PROYECTO COMPLETO · FASES 1, 2 Y 3

# Gestión de Incidencias con Power Platform e IA

De la solución base al RAG, la evaluación, la operación en Azure y la hoja de ruta a producción

Power Apps · Dataverse · Power Automate · Teams · Power BI · Azure OpenAI · Azure AI Search · Azure Key Vault · Entra ID · Log Analytics

**José Rafael Vázquez Silva**
Proyecto personal de portafolio
github.com/RafaVazquezSilva/powerplatform-gestion-incidencias
Octubre de 2026

*AudioVital es una marca ficticia. Todos los clientes, incidencias y documentos son inventados para este proyecto y no corresponden a ninguna empresa real.*

# Presentación del proyecto

Este documento reúne el proyecto completo: un circuito de gestión de incidencias de atención al cliente para **AudioVital**, una marca ficticia de audífonos, construido y verificado en un entorno real de Microsoft Power Platform y Azure con datos inventados. Se desarrolló en tres fases, y cada una se apoya en la anterior.

## El problema

- Las incidencias llegan en texto libre, por tienda, teléfono o correo, y alguien tiene que decidir qué son y cuánta prisa corren.
- Las urgentes (pérdida total de audición, cita médica inminente, cobro duplicado) no pueden quedarse en una bandeja.
- Los agentes consultan políticas de garantías, reclamaciones o citas en documentos que nadie se sabe de memoria.
- No queda claro quién lleva cada caso ni cómo se resolvió.

## La solución, en una frase por rol

- **Agente:** registra la incidencia con sus palabras; la IA la clasifica por tipo y prioridad en 2–3 segundos y el asistente de Teams le responde con la política vigente, citando el documento.
- **Supervisor:** recibe las urgentes al momento en Teams y se las asigna con el botón «Me encargo».
- **Cierre:** ninguna incidencia se cierra sin resolución escrita, y el autor recibe el aviso.
- **Dirección e IT:** panel en Power BI, claves protegidas, permisos mínimos, costes con presupuesto y errores visibles en Log Analytics.

## Las tres fases

| Fase | Objetivo | Qué se construyó | Resultado clave |
|---|---|---|---|
| **1 · Solución base** | Circuito completo de punta a punta | Dataverse (3 tablas), alta en Power Apps, Flujo 1 de clasificación con Azure OpenAI (gpt-4.1-mini) y escalado por Teams, asistente en Teams, panel en Power BI | Clasificación en 2–3 s por incidencia |
| **2 · RAG y evaluación** | Respuestas fundamentadas y clasificación medible | Base documental de 6 PDFs indexada en Azure AI Search (11 fragmentos, text-embedding-3-small, búsqueda híbrida); temperatura 0, salida JSON y rama de error; evaluación con 45 casos | 80 % estricto en validación independiente y ningún escalado perdido |
| **3 · Operación y gobierno** | Que un equipo de IT pudiera operarlo | Key Vault con entidad de servicio y RBAC; presupuesto con alertas; diagnósticos en Log Analytics (KQL); corrección del bucle de autodisparo; Flujo 3 (cierre con resolución) y Flujo 4 («Me encargo») | Respuestas del asistente de 5 min a 8–15 s y −80 % de carga por consulta |

# Índice general

## Cómo leer este documento

Une las dos memorias técnicas sin cambios: la Parte I cubre las Fases 1 y 2, y la Parte II la Fase 3 con la **hoja de ruta para llevarlo a producción**. Cada parte conserva su portada y su numeración interna; las páginas de este índice son las del PDF completo, y los marcadores del lector llevan a cada sección. Capturas y datos de evaluación: repositorio de GitHub de la portada.

**Presentación del proyecto ..... 2**

**Parte I — Memoria técnica de las Fases 1 y 2 ..... 4**

1. Resumen ejecutivo ..... 6
2. Contexto y objetivo ..... 7
3. Evolución del proyecto: de la Fase 1 a la Fase 2 ..... 8
4. Arquitectura ..... 9
5. Fase 1 — Solución base ..... 10
6. Fase 2 — Base documental y Azure AI Search ..... 12
7. Fase 2 — Asistente con respuestas fundamentadas (RAG) ..... 13
8. Fase 2 — Clasificación robusta (Flujo 1) ..... 14
9. Evaluación de la clasificación ..... 16
10. Incidencia en producción: cuota semántica agotada ..... 17
11. Problemas resueltos ..... 18
12. Límites conocidos ..... 19
13. Evidencias ..... 20
14. Continuación ..... 24

**Parte II — Memoria técnica de la Fase 3 ..... 25**

1. Resumen ejecutivo ..... 27
2. Punto de partida y objetivo de la Fase 3 ..... 28
3. Modelo de uso: quién usa y quién gestiona la solución ..... 29
4. Arquitectura final (Fases 1 a 3) ..... 30
5. Seguridad: Key Vault, entidad de servicio y RBAC ..... 31
6. Observabilidad y control de costes ..... 33
7. Incidente: bucle de autodisparo del Flujo 2 ..... 35
8. Ciclo de vida de la incidencia en Dataverse ..... 38
9. Flujo 3 — Cierre con resolución obligatoria ..... 39
10. Flujo 4 — Asignación «Me encargo» con tarjeta adaptable ..... 41
11. Problemas resueltos en la Fase 3 ..... 43
12. Lo que no se construyó ..... 44
13. Límites conocidos (consolidados) ..... 45
14. Hoja de ruta para llevarlo a producción ..... 46
15. Evidencias ..... 49

MEMORIA TÉCNICA

# Gestión de Incidencias con Power Platform e IA

Fase 1 y Fase 2: de la solución base al asistente con conocimiento corporativo

Power Apps · Dataverse · Power Automate · Azure OpenAI · Azure AI Search · Microsoft Teams · Power BI

**José Rafael Vázquez Silva**
Proyecto personal de portafolio
Octubre de 2026

*AudioVital es una marca ficticia. Todos los clientes, incidencias y documentos son inventados para este proyecto y no corresponden a ninguna empresa real.*

# Índice

1. Resumen ejecutivo..... 3
2. Contexto y objetivo..... 4
3. Evolución del proyecto: de la Fase 1 a la Fase 2..... 5
4. Arquitectura..... 6
5. Fase 1 — Solución base..... 7
6. Fase 2 — Base documental y Azure AI Search..... 9
7. Fase 2 — Asistente con respuestas fundamentadas (RAG)..... 10
8. Fase 2 — Clasificación robusta (Flujo 1)..... 11
9. Evaluación de la clasificación..... 13
10. Incidencia en producción: cuota semántica agotada..... 14
11. Problemas resueltos..... 15
12. Límites conocidos..... 16
13. Evidencias..... 17
14. Continuación..... 21

# 1. Resumen ejecutivo

Este documento describe un proyecto personal de portafolio construido íntegramente sobre **Microsoft Power Platform** y **Azure**: un sistema de gestión de incidencias de atención al cliente para una marca ficticia de audífonos, **AudioVital**. Ningún dato, cliente ni marca corresponde a una empresa real.

El proyecto se desarrolló en dos fases. La **Fase 1** construyó la solución base: alta de incidencias en Power Apps, clasificación automática por tipo y prioridad con Azure OpenAI, escalado al supervisor por Microsoft Teams, un asistente conversacional en Teams y un panel en Power BI. La **Fase 2** la llevó hacia una arquitectura más cercana a la empresarial: el asistente pasa a responder **fundamentado en documentación corporativa y citando la fuente** (RAG con Azure AI Search), la clasificación se vuelve **determinista, estructurada y con ruta de error**, y su calidad se **mide con un conjunto de evaluación** en lugar de comprobarse con unos pocos casos.

| Indicador | Fase 1 | Fase 2 |
|---|---|---|
| Respuestas del asistente | De memoria del modelo | Fundamentadas en 6 documentos, con cita de la fuente |
| Clasificación | Temperatura 1, salida de texto | Temperatura 0, JSON obligatorio, criterios documentales |
| Si la clasificación falla | La incidencia queda sin clasificar y nadie se entera | Aviso automático en Teams (verificado con fallo forzado) |
| Verificación | 4 casos de comprobación | 45 incidencias evaluadas en 3 pasadas |
| Acierto en incidencias nuevas | No medido | **80 % estricto · 100 % en casos claros · 0 escalados perdidos** |
| Coste en Azure | 0,07 € | Capas gratuitas; coste marginal |

Además de construir, el proyecto documenta lo que **no funcionó**: errores encontrados y resueltos, un fallo de la propia documentación de la Fase 1 detectado gracias a la evaluación, una incidencia real en producción provocada por una cuota de la capa gratuita y una sección de **límites conocidos**.

# 2. Contexto y objetivo

El proyecto nace de un hueco concreto en el perfil profesional del autor: experiencia demostrable en **Microsoft Power Platform**, frente a un portafolio previo centrado en Make, n8n y la API de OpenAI. Se eligió un proceso que el autor conoce de primera mano —la gestión de incidencias de atención al cliente— y se decidió construirlo de principio a fin, documentarlo con rigor y publicarlo, en lugar de acumular tutoriales.

Tras completar la Fase 1 se tomó una decisión de enfoque: **profundizar en el mismo caso de uso** antes de abordar otros. Evolucionar una solución —medir, encontrar sus fallos y corregirlos— demuestra más criterio que sumar proyectos pequeños que solo conectan servicios. La Fase 2 responde a una pregunta concreta: *¿cómo se convierte un prototipo funcional en algo más cercano a una arquitectura empresarial de IA?*

## 2.1 Principios de trabajo

- **Marca y datos ficticios siempre.** Ningún nombre de empresa real.
- **Paso a paso, con verificación.** Cada cambio se comprueba en el entorno real y se recoge evidencia.
- **Nada de sobreventa.** Se presenta como lo que es: un portafolio personal, construido íntegramente por el autor, con sus límites declarados.

# 3. Evolución del proyecto: de la Fase 1 a la Fase 2

Esta sección explica el punto de partida, qué carencias se identificaron al cerrar la Fase 1 y cómo la Fase 2 respondió a cada una.

## 3.1 Dónde terminó la Fase 1

Al cierre de la Fase 1 (24 de septiembre de 2026) el sistema funcionaba de extremo a extremo: una incidencia creada en Power Apps se clasificaba en 2–3 segundos, se escalaba por Teams si era de prioridad Alta y aparecía en el panel de Power BI. Un asistente en Teams respondía preguntas sobre las incidencias registradas. La solución se publicó en GitHub con memoria técnica y capturas.

## 3.2 Carencias identificadas al cerrarla

| Carencia de la Fase 1 | Por qué importaba | Respuesta en la Fase 2 |
|---|---|---|
| El asistente respondía de memoria | Ante preguntas de procedimiento podía inventar respuestas plausibles | RAG con Azure AI Search sobre 6 documentos; cita de la fuente; instrucción explícita de no inventar |
| Clasificación no determinista | Temperatura 1: la misma incidencia podía clasificarse distinto | Temperatura 0 y `response_format = json_object` |
| Prompt sin criterios | El modelo decidía la prioridad con su propio juicio | Criterios de Tipo y Prioridad tomados literalmente de la documentación |
| Sin ruta de error | Un fallo dejaba la incidencia sin clasificar en silencio | Rama paralela que avisa por Teams; verificada con fallo forzado |
| Verificación con 4 casos | Una comprobación, no una evaluación | Conjunto de 30 casos, segunda pasada y validación independiente de 15 |

## 3.3 Lo que la Fase 2 descubrió de la Fase 1

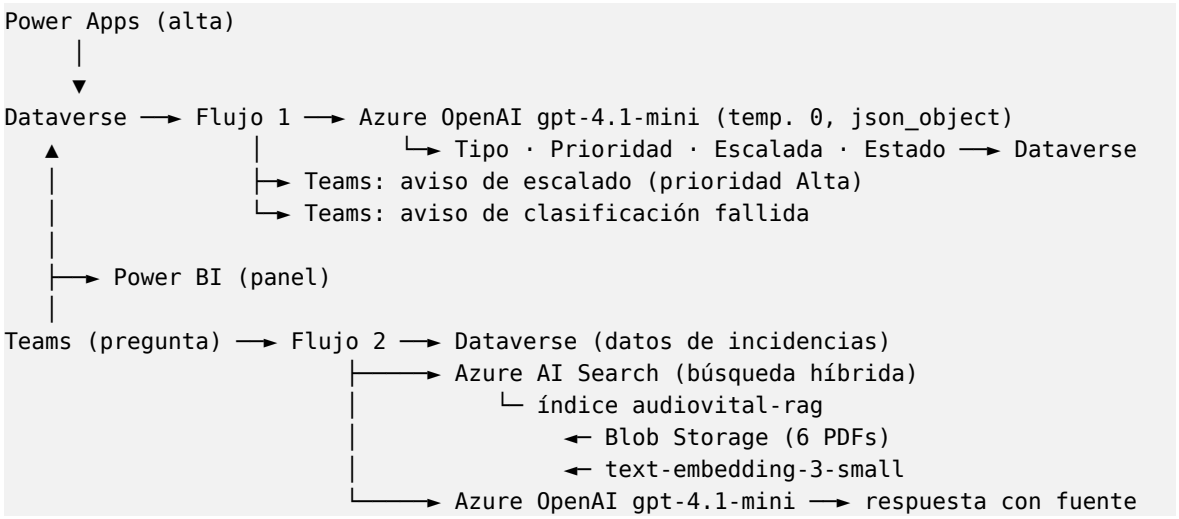
Medir sirvió también para revisar lo que ya se daba por bueno. Dos hallazgos corrigen afirmaciones de la Fase 1:

- **El acierto del caso del pitido fue en parte azar.** Con temperatura 0 la misma descripción se clasificó como Media en lugar de Baja; el prompt no contenía criterios (sección 8.2).
- **La rama de escalado nunca cambiaba el Estado.** La memoria de la Fase 1 afirmaba que lo ponía en "Escalada"; la evaluación mostró que solo enviaba el aviso por Teams. Se corrigió el flujo y se deja constancia aquí del error de documentación (sección 8.5).

> *Declarar estos errores es deliberado: un portafolio que solo muestra aciertos no permite juzgar el criterio de quien lo construyó.*

# 4. Arquitectura

La arquitectura final mantiene la base de la Fase 1 y añade la capa documental de la Fase 2.




| Capa | Tecnología | Función | Fase |
|---|---|---|---|
| Datos | Microsoft Dataverse | Fuente única de verdad; autonumeración INC-0001… | 1 |
| Captura | Power Apps | Formulario de alta y consulta | 1 |
| Orquestación | Power Automate | Clasificación, escalado, asistente, avisos de error | 1 y 2 |
| Inteligencia | Azure OpenAI · gpt-4.1-mini | Clasificación y generación de respuestas | 1 y 2 |
| Embeddings | Azure OpenAI · text-embedding-3-small | Vectorización de documentos y consultas | 2 |
| Base documental | Azure Blob Storage | 6 PDFs de políticas y protocolos | 2 |
| Recuperación | Azure AI Search (plan Gratis) | Índice vectorizado y búsqueda | 2 |
| Interacción | Microsoft Teams | Avisos y asistente conversacional | 1 y 2 |
| Analítica | Power BI | Panel interactivo con medida DAX | 1 |

# 5. Fase 1 — Solución base

## 5.1 Dataverse

Tres tablas relacionadas: `Clientes`, `Productos_Servicios` e `Incidencias` (nombre lógico `cr3f3_incidencias`). La columna `Título_Incidencia` usa autonumeración con prefijo `INC` y cuatro dígitos.

Las columnas de opción almacenan enteros, no texto, y esto condiciona todas las expresiones de los flujos:

| Columna | Opción | Valor |
|---|---|---|
| Tipo | Reclamación · Avería · Otro | 610150000 · 610150001 · 610150002 |
| Prioridad | Alta · Media · Baja | 610150000 · 610150001 · 610150002 |
| Estado | Abierta · En curso · Escalada · Cerrada | 610150000 · 610150001 · 610150002 · 610150003 |

## 5.2 Flujo 1 — Alta de incidencia

Se dispara al crear una fila en `Incidencias`. Envía la descripción a Azure OpenAI, que devuelve un JSON con tipo y prioridad. Como la acción devuelve un array, los pasos siguientes quedan dentro de un **For each**. Los valores se convierten a los enteros de Dataverse con expresiones directas:

```
if(contains(toLower(json(items('For_each')?['message']?['content'])?
['prioridad']),'alta'),610150000,
   if(contains(toLower(json(items('For_each')?['message']?['content'])?
['prioridad']),'media'),610150001,610150002))
```

Si la prioridad es Alta, una **Condición** envía un aviso al supervisor por Teams.

## 5.3 Flujo 2 — Asistente en Teams

Se dispara cuando un mensaje del canal contiene la palabra clave "Incidencias". Como el desencadenador solo devuelve identificadores, se añade *Obtener detalles del mensaje* para leer el texto. El flujo lee las incidencias de Dataverse y pide a Azure OpenAI una respuesta. El mensaje de sistema prohíbe expresamente mostrar campos internos (`statecode`, `statuscode`, GUID).

## 5.4 Power BI

Power BI Desktop conectado a Dataverse (modo Importar), con la medida `Total incidencias = COUNTROWS(cr3f3_incidencias)` y cuatro visualizaciones con filtrado cruzado: por tipo, prioridad, canal y estado.

## 5.5 Restricciones del entorno

El entorno impuso rediseños que forman parte del valor del proyecto:

| Restricción | Solución aplicada |
|---|---|
| SharePoint no disponible en el inquilino | Persistencia sobre tablas nativas de Dataverse |
| Microsoft Forms no habilitado | Aplicación de Power Apps + desencadenador de creación de fila |
| Copilot Studio sin créditos | Asistente reconstruido con Power Automate + Teams + Azure OpenAI. |

| Restricción | Solución aplicada |
|---|---|
| | El agente llegó a crearse con el servidor MCP de Dataverse conectado; la ejecución quedó bloqueada por un límite de licencia (`EnforcementUsageCredits`) |
| Modelo sin cuota en la región | Despliegue de `gpt-4.1-mini` |
| Directorio de Azure ajeno (sin rol de administrador del inquilino) | No se pueden crear usuarios en Entra ID; previsto sustituir por roles de Dataverse y RBAC por recurso |

## 5.6 Verificación de la Fase 1

| Descripción | Esperado | Resultado |
|---|---|---|
| Cobro duplicado, exige devolución | Reclamación / Alta | Correcto |
| Pitido intermitente sin pérdida de función | Avería / Baja | Correcto* |
| Petición de cambio de cita | Otro / Baja | Correcto |
| Pérdida total de audición, cita médica inminente | Avería / Alta + escalado | Correcto |

> ** En la Fase 2 se comprobó que, con temperatura 0 y sin criterios en el prompt, este caso se clasificaba como Media. El acierto original fue en parte azar (sección 8.2). Cuatro casos son una comprobación, no una evaluación: esa fue la carencia que motivó el punto F de la Fase 2.*

# 6. Fase 2 — Base documental y Azure AI Search

## 6.1 Recursos creados

| Recurso | Nombre | Configuración clave |
|---|---|---|
| Storage Account | `audiovitalstorage` | Estándar, redundancia LRS (no GRS, por coste) |
| Contenedor | `base-documental` | Privado, sin acceso anónimo |
| Azure AI Search | `audiovital-search` | Plan Gratis (0 $/mes), East US |
| Índice | `audiovital-rag` | 11 fragmentos · 248 KB · 200 KB de vectores (de 50 MB) |
| Indexador | `audiovital-rag-indexer` | Ejecución única: 6 documentos, 0 errores, 3 s |
| Modelo de embeddings | `text-embedding-3-small` | En el recurso de Azure OpenAI existente; Estándar global |

## 6.2 Base documental

Seis documentos internos ficticios de AudioVital, redactados para el proyecto y **coherentes con la lógica de clasificación**, de modo que el asistente y el clasificador se rigen por la misma fuente:

| Documento | Contenido |
|---|---|
| `procedimiento_averias.pdf` | Criterios de prioridad de averías (Alta, Media, Baja) |
| `politica_reclamaciones.pdf` | Prioridades de reclamación; umbral de 150 € para aprobación de supervisor |
| `protocolo_escalado.pdf` | Reglas de escalado y plazos del supervisor |
| `cambios_citas.pdf` | Cambios y cancelaciones de citas |
| `garantia_productos.pdf` | Garantías por categoría y exclusiones |
| `protocolo_atencion_cliente.pdf` | Protocolo general; prohibición de exponer campos internos |

## 6.3 Creación del índice

Se usó el asistente *Importación de datos* de Azure AI Search en el escenario **RAG**: origen Blob Storage, vectorización con `text-embedding-3-small` a través del recurso de Azure OpenAI, sin procesamiento de imágenes, clasificador semántico habilitado e indexación única (los documentos son fijos).

**Errores resueltos:** el asistente no encontraba un modelo compatible porque solo existía `gpt-4.1-mini`, un modelo de chat; hubo que desplegar el modelo de embeddings. La redundancia del almacenamiento venía por defecto en GRS y se cambió a LRS antes de crear.

**Verificación:** la consulta *"el cliente perdió completamente la audición y tiene cita médica mañana"* devolvió en primer lugar `protocolo_escalado.pdf`, con puntuación 0,979.

# 7. Fase 2 — Asistente con respuestas fundamentadas (RAG)

## 7.1 Cambios en el Flujo 2

Se insertó un paso **HTTP** entre la lectura de Dataverse y la llamada a Azure OpenAI, que consulta el índice con el texto del mensaje del cliente. Configuración vigente:

```
POST https://audiovital-search.search.windows.net/indexes/audiovital-rag
     /docs/search?api-version=2024-07-01
Content-Type: application/json
api-key: <clave de consulta, solo lectura>
{
  "search": "<texto del mensaje>",
  "vectorQueries": [
    { "kind": "text", "text": "<texto del mensaje>",
      "fields": "text_vector", "k": 8 }
  ],
  "top": 8,
  "select": "title,chunk"
}
```

Con `kind: "text"`, Azure AI Search vectoriza la consulta con el mismo modelo del índice, sin llamada adicional desde el flujo. La configuración original usaba el clasificador semántico con `top: 3`; el motivo del cambio se explica en la sección 10.

Los fragmentos recuperados se pasan al modelo en un mensaje adicional, y el mensaje de sistema incorpora tres instrucciones: basarse en ellos para preguntas de procedimiento, **citar el documento de origen** y, si la información no está ni en los fragmentos ni en los datos, **decirlo en lugar de inventar**.

## 7.2 Pruebas funcionales en Teams

| Pregunta | Resultado |
|---|---|
| Pérdida total de audición y cita médica mañana | Correcta; cita el Protocolo de Escalado |
| Duración y cobertura de la garantía | Correcta; cita `garantia_productos.pdf` |
| Cancelar una cita de revisión | Correcta según `cambios_citas.pdf` |
| Horario de atención los domingos (no está en la documentación) | **No inventa**: indica que no dispone de esa información y remite a atención directa |
| Pérdida total de audición (con búsqueda híbrida) | Correcta y completa; cita el Procedimiento de Averías en vez del Protocolo de Escalado |

> *La prueba fuera de la documentación es la más relevante: confirma que el sistema reconoce los límites de su conocimiento en lugar de generar una respuesta plausible pero falsa.*

# 8. Fase 2 — Clasificación robusta (Flujo 1)

## 8.1 Determinismo y salida estructurada

| Parámetro | Fase 1 | Fase 2 | Motivo |
|---|---|---|---|
| `temperature` | 1 | **0** | Misma entrada, misma salida |
| `response_format` | `text` | **`json_object`** | JSON siempre válido |

El Flujo 2 conserva su configuración conversacional: los dos usos del modelo —clasificar y conversar— quedan separados también en los parámetros.

## 8.2 Criterios documentales en el prompt

Con temperatura 0, *"pitido intermitente desde ayer, pero sigue oyendo bien"* se clasificó como **Media**; según la política es **Baja**. La causa: el prompt solo enumeraba los valores posibles. Se reescribió con las definiciones de cada Tipo y los criterios de Prioridad copiados de los documentos, y la misma descripción pasó a clasificarse correctamente. Tras la primera evaluación se añadieron dos reglas más (sección 9).

```
PRIORIDAD — Avería:
- Alta: pérdida total o severa de audición, o cita médica / evento impostergable en
48 horas.
- Media: fallo parcial que reduce la calidad pero permite el uso básico.
- Baja: fallo intermitente, estético o que no afecta a la audición.
(… criterios equivalentes para Reclamación y Otro …)
Si el cliente indica que sigue oyendo bien, la prioridad de una avería no puede ser
Alta.
La pérdida total de audición es siempre prioridad Alta, aunque el cliente no exprese
urgencia.
Cualquier queja sobre un cobro, aunque sea pequeño, es como mínimo prioridad Media.
```

## 8.3 Ruta de error

Se añadió una rama paralela que publica en Teams *"Clasificación automática fallida. Revisar manualmente la incidencia: INC-XXXX"* cuando la actualización **falla, se omite o agota el tiempo**. "Se omite" cubre el caso en que falla el análisis del JSON y la actualización ni siquiera llega a ejecutarse.

**Verificación con fallo forzado:** se introdujo temporalmente un valor inexistente (999) en el Estado. Dataverse rechazó la actualización completa (la operación es atómica), el aviso llegó a Teams para INC-0010 y, tras revertir el cambio, una prueba de regresión (INC-0011) confirmó que el camino normal seguía funcionando.

## 8.4 Estado por defecto

Durante las pruebas el autor detectó que las incidencias no escaladas quedaban **sin Estado**. Se resolvió en la capa de datos —opción predeterminada "Abierta" en la columna— para que toda incidencia nazca con un estado válido aunque el flujo falle, como se comprobó con INC-0010.

## 8.5 Estado "Escalada" (corrección de la Fase 1)

La evaluación reveló que las incidencias Alta quedaban con `Escalada_a_Supervisor = Sí` pero Estado "Abierta". La rama de escalado solo enviaba el aviso. Se añadió el Estado a la misma

actualización que escribe Tipo y Prioridad, con la misma lógica que la marca de escalado, de modo que no pueden contradecirse:

```
if(contains(toLower(json(items('For_each')?['message']?['content'])?
['prioridad']),'alta'),
   610150002,   // Escalada
   610150000)   // Abierta
```

Resultado: Estado coherente con la prioridad en **45 de 45** incidencias evaluadas.

# 9. Evaluación de la clasificación

## 9.1 Diseño

- **Conjunto principal:** 30 incidencias (12 Avería, 9 Reclamación, 9 Otro): 25 claras, 3 de frontera y 2 ambiguas.
- **Resultado esperado fijado antes de ejecutar**, citando el documento y criterio de origen. Los casos con dos lecturas defendibles llevan una alternativa aceptable.
- **Carga masiva:** CSV importado en Dataverse; el Flujo 1 se dispara por cada fila; los resultados se exportan a Excel y se comparan con fórmulas.
- **Métrica prioritaria: escalados perdidos.** Una incidencia Alta clasificada por debajo no llega al supervisor; es el error con más coste.

## 9.2 Resultados

| Pasada | Incidencias | Acierto estricto | Con alternativas | Escalados perdidos |
|---|---|---|---|---|
| v1 | 30 (INC-0012 a 0041) | 27/30 (90 %) | 28/30 | 1 |
| v2 · tras corregir | mismas 30 (INC-0042 a 0071) | 29/30 | 30/30 · 0 regresiones | 0 |
| **Validación independiente** | **15 nuevas** (INC-0072 a 0086) | **12/15 (80 %)** | 15/15 | **0** |

## 9.3 Fallos y correcciones

La v1 falló en dos casos. *"No oye nada, pero no tiene prisa porque está de viaje"* se clasificó como Media en lugar de Alta: el modelo rebajó la prioridad por la falta de urgencia expresada, y fue un **escalado perdido**. *"Me han cobrado 4 € de más; no pido nada"* salió Baja en lugar de Media. Se añadieron dos reglas al prompt y la v2 corrigió ambos casos sin empeorar ninguno.

## 9.4 Por qué hacía falta una validación independiente

Las reglas de la v2 se escribieron después de ver los fallos de la v1 en ese mismo conjunto, así que su 100 % demuestra que la corrección funciona, pero **no estima** el acierto con incidencias nuevas. Para eso se redactaron 15 incidencias que no se usaron para ajustar nada, con texto más realista: sin tildes, mensajes escuetos, una felicitación en lugar de una queja y una petición que mezcla cita y avería.

**Resultado: 80 % estricto, 100 % en los casos claros y ningún escalado perdido.** Las tres desviaciones están en casos marcados de antemano como discutibles: un audífono sin sonido y el otro funcionando (escaló de más, que es el error tolerable), una queja sobre la garantía clasificada como Baja en lugar de Media, y una felicitación clasificada como Reclamación porque la política no prevé opiniones positivas.

> *Los errores aparecen donde la propia política es ambigua, no en la comprensión del texto. La mejora del prompt que lo resolvería no se aplicó, para no volver a ajustar sobre datos ya vistos.*

# 10. Incidencia en producción: cuota semántica agotada

Al recoger las capturas finales, el explorador de búsqueda devolvió: *"Free Query Semantic Usage exceeded for the month"*. El plan Gratis de Azure AI Search limita las consultas mensuales con clasificador semántico, y la cuota se había consumido. Como el Flujo 2 usaba ese clasificador, **todas las consultas del asistente fallaban, sin ningún aviso**.

## 10.1 Diagnóstico

Repitiendo la consulta de referencia sin clasificador semántico (búsqueda híbrida por palabras clave y vectores):

| Posición | Documento | Puntuación |
|---|---|---|
| 1 | procedimiento_averias | 0,652 |
| 2 | procedimiento_averias | 0,642 |
| 3 | cambios_citas | 0,638 |
| 4–6 | averías, garantía, atención al cliente | 0,635 – 0,623 |
| **7** | **protocolo_escalado** | **0,615** |

Sin la reordenación semántica las puntuaciones quedan casi empatadas y el documento más relevante cae al séptimo puesto: con el `top: 3` original, el asistente **no habría recibido el protocolo de escalado**.

## 10.2 Decisión

| Opción | Valoración |
|---|---|
| Activar la facturación semántica | No disponible en el plan Gratis; exige subir de plan y el crédito caducaba en días |
| Esperar al reinicio mensual de la cuota | El asistente quedaría roto entretanto y el problema se repetiría |
| **Búsqueda híbrida y más fragmentos** | **Elegida.** Con 11 fragmentos pequeños, pedir 8 cuesta pocos tokens y garantiza que el documento relevante llega al modelo |

**Verificación:** la misma pregunta en Teams obtuvo una respuesta correcta y completa, con datos que solo están en el protocolo de escalado. La diferencia: atribuye la fuente al primer documento del ranking. **La respuesta es correcta; la cita es menos precisa.**

> *Lección: una dependencia de la capa gratuita puede romper el sistema sin aviso. La solución elegida funciona porque el corpus es pequeño y no escalaría a cientos de documentos.*

# 11. Problemas resueltos

| Fase | Problema | Solución |
|---|---|---|
| 1 | Las salidas de *Analizar JSON* no aparecían en el contenido dinámico | Expresiones directas sobre la respuesta del modelo |
| 1 | *Actualizar una fila* fuera del For each | Recolocada dentro del bucle |
| 1 | Las columnas de opción rechazaban texto | Conversión a enteros con `if/contains` |
| 1 | `INC--0006` con doble guion | El prefijo debía ser `INC`, no `INC-` |
| 1 | Teams: *Subscription Id(s) are not found* | Reseleccionar el canal y guardar para regenerar la suscripción |
| 1 | El asistente ignoraba la pregunta | Añadir *Obtener detalles del mensaje* |
| 1 | El asistente mostraba "estado Activo" | Prohibir en el prompt los campos internos de Dataverse |
| 2 | Ningún modelo compatible para vectorizar | Desplegar `text-embedding-3-small` |
| 2 | `queryLanguage` no soportado por la versión de la API | Eliminar el parámetro |
| 2 | Rol vacío en el tercer mensaje al guardar | Seleccionar `user` explícitamente |
| 2 | Clasificación inestable y sin criterios | Temperatura 0, `json_object` y criterios documentales |
| 2 | Incidencias sin Estado | Opción predeterminada "Abierta" en Dataverse |
| 2 | Estado no pasaba a "Escalada" | Estado escrito por expresión en la misma actualización |
| 2 | Escalado perdido y cobro menor mal clasificados | Dos reglas en el prompt; corregidos sin regresiones |
| 2 | Importar desde Excel rechazaba el archivo | Usar *Importar desde CSV* |
| 2 | Cuota semántica agotada | Búsqueda híbrida y `top` 8 |

# 12. Límites conocidos

Esta sección declara lo que la solución **no** hace o no se ha probado. Es parte del entregable, no una nota al margen.

1. **Dependencia de la capa gratuita.** La cuota semántica se agotó sin aviso y dejó el asistente inoperativo; no había monitorización de errores del Flujo 2 y se detectó por casualidad.
2. **Recuperación sin reordenación semántica.** Funciona porque el corpus tiene 11 fragmentos; no escala a una base documental real.
3. **Cita de la fuente menos precisa** en modo híbrido: la respuesta puede ser correcta pero atribuirse a otro documento.
4. **Evaluación pequeña** (30 + 15 casos), redactada y etiquetada por una sola persona, sin segundo anotador.
5. **Huecos en la política documental:** no hay categoría para opiniones positivas y la frontera Media/Baja en quejas contractuales es ambigua.
6. **Clave de API** en el paso HTTP en lugar de identidad administrada y Azure Key Vault.
7. **El fallo forzado fue de escritura en Dataverse**, no de un JSON inválido del modelo, que `json_object` hace muy improbable.
8. **Copilot Studio** no pudo ejecutarse por límite de licencia; el asistente se reconstruyó con Power Automate.
9. **Infraestructura temporal:** el crédito de Azure caduca hacia el 21 de octubre de 2026; después, la solución queda documentada pero no ejecutable sin pasar a pago por uso.

## 12.1 Qué se haría distinto con presupuesto real

- Plan de pago de Azure AI Search con clasificador semántico y alertas de cuota.
- Identidad administrada, secretos en Key Vault y control de acceso por roles.
- Registro de cada ejecución (modelo, tokens, latencia, errores) y alertas sobre fallos de los flujos.
- Evaluación con incidencias reales anonimizadas y varios anotadores.

# 13. Evidencias

Selección de capturas del entorno real. El conjunto completo (29 capturas) se encuentra en la carpeta `capturas/fase2/` del repositorio; la clave de API aparece tapada.

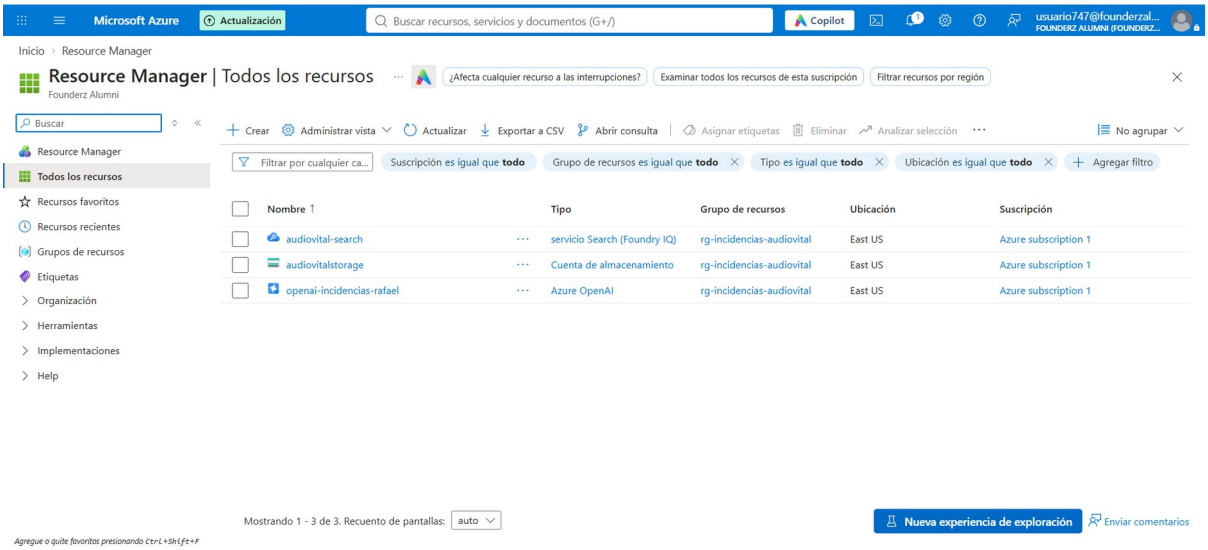



**Figura 1.** *Recursos de Azure de la Fase 2 en el grupo rg-incidencias-audiovital.*

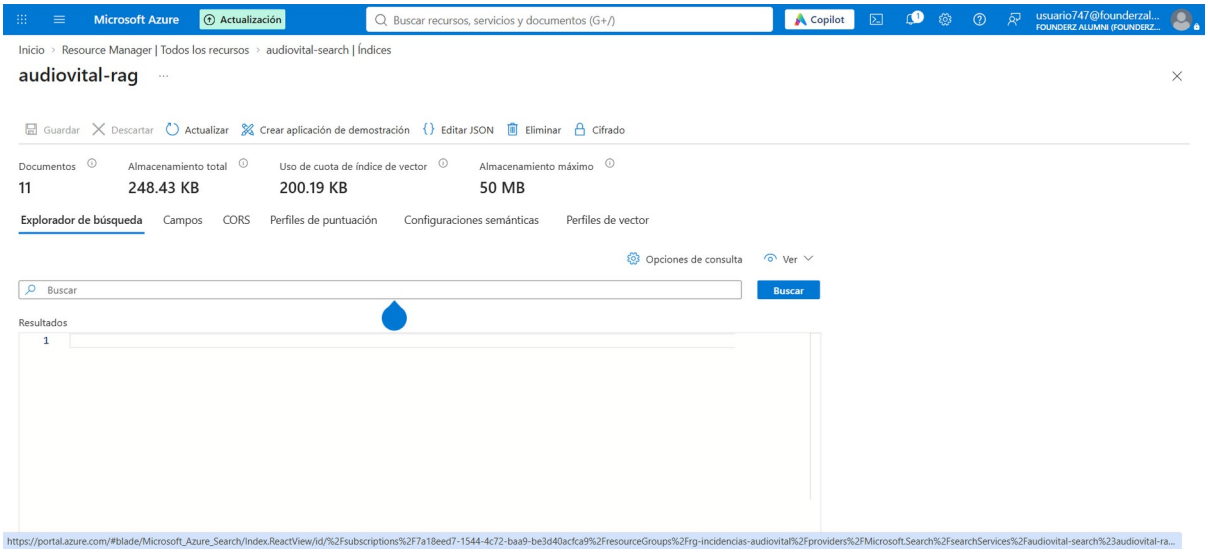



**Figura 2.** *Índice audiovital-rag: 11 fragmentos y 200 KB de vectores.*

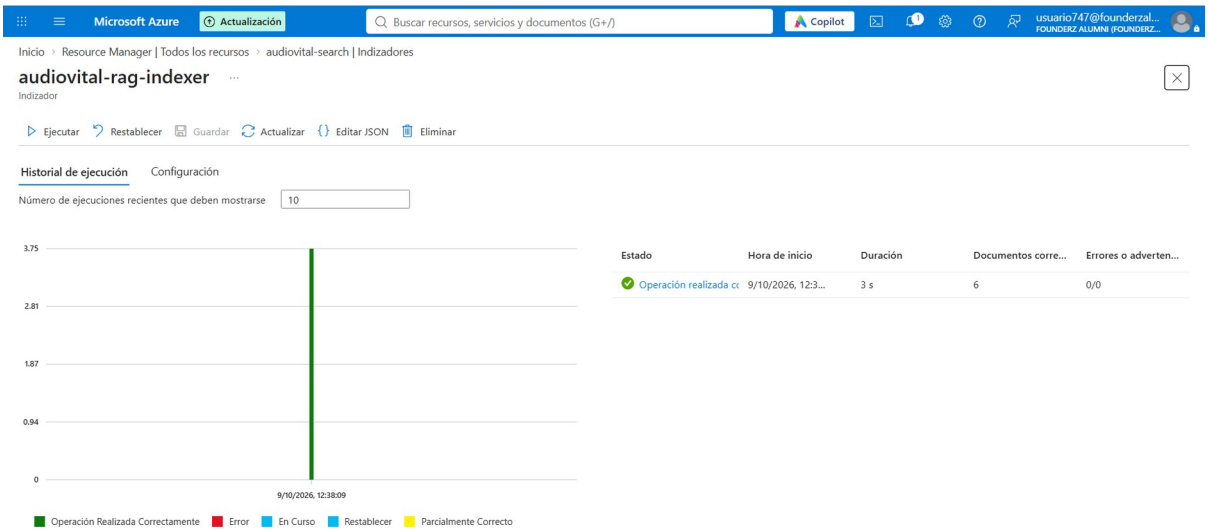



**Figura 3.** *Indexador: 6 documentos procesados sin errores.*

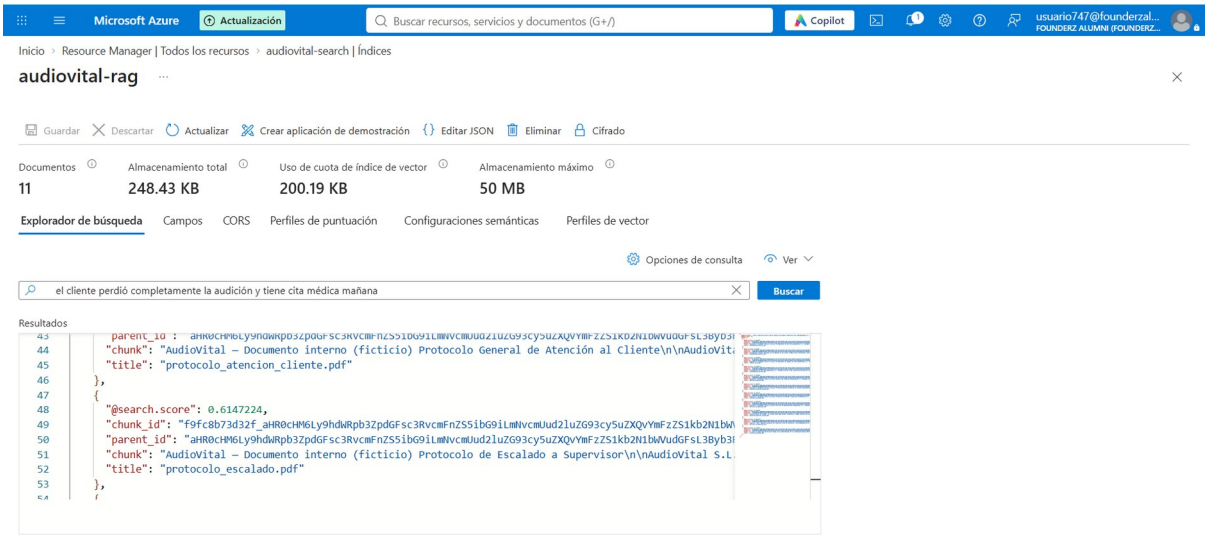



**Figura 4.** *Búsqueda híbrida: el protocolo de escalado aparece en séptima posición.*

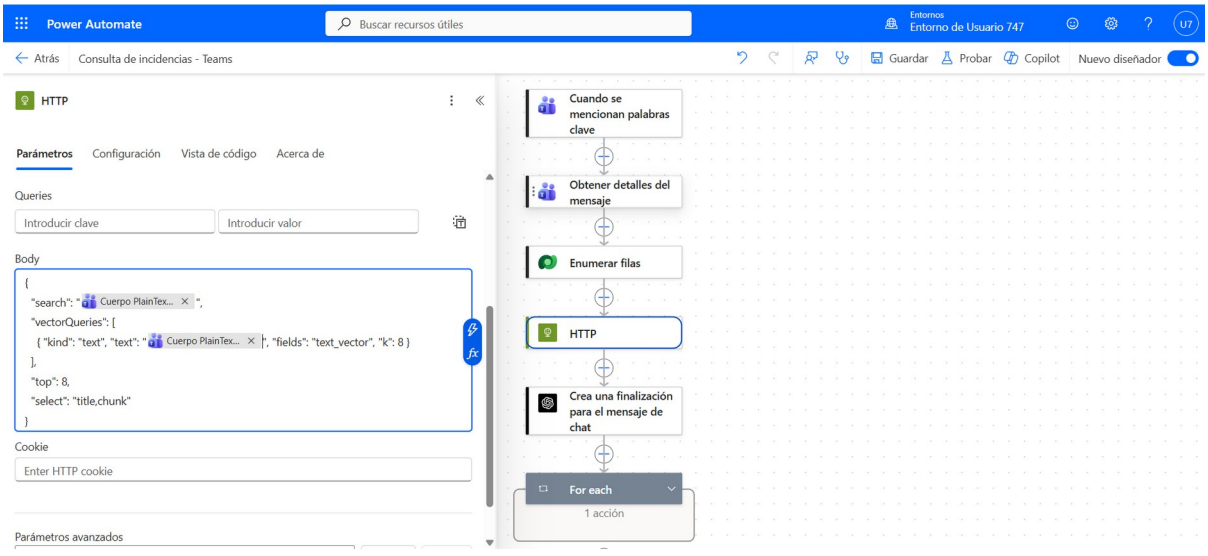



**Figura 5.** *Flujo 2: cuerpo de la petición de búsqueda híbrida.*

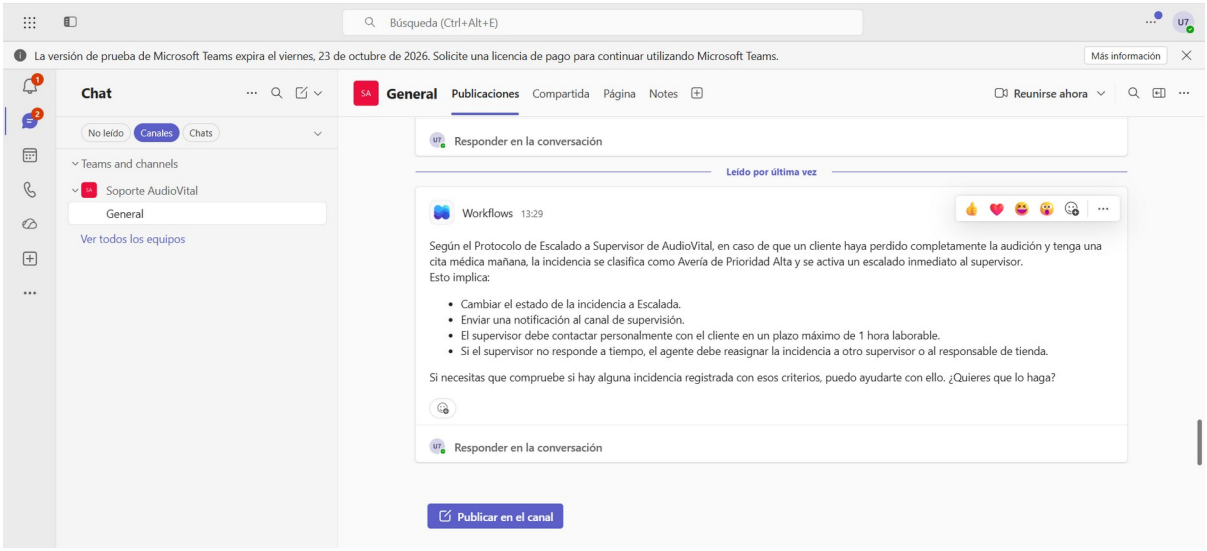



**Figura 6.** *Respuesta del asistente fundamentada en el protocolo de escalado.*

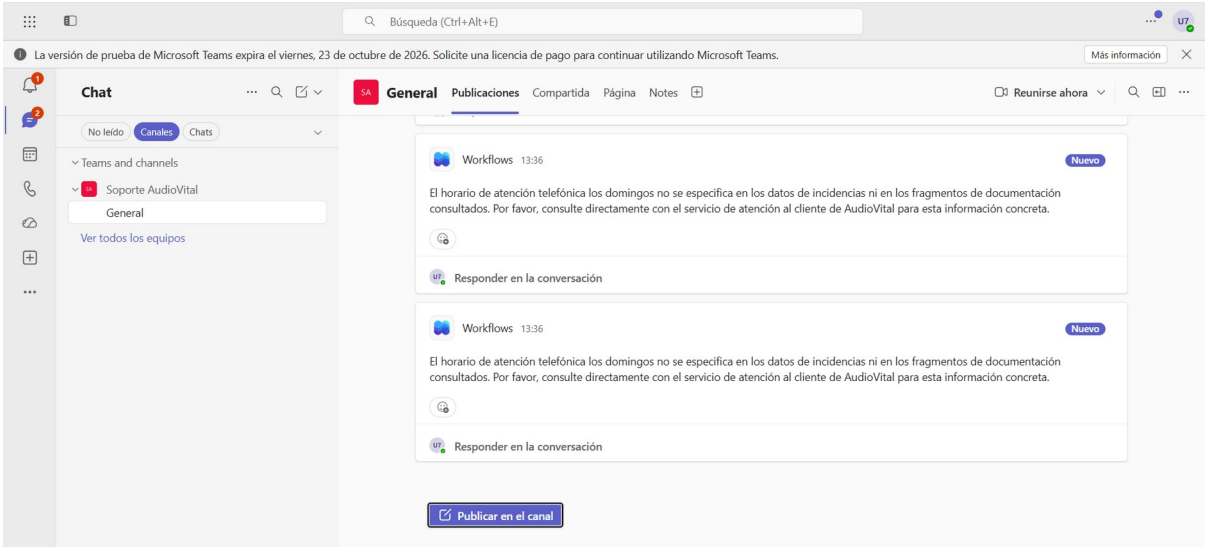



**Figura 7.** *Ante una pregunta sin respaldo documental, el asistente no inventa.*

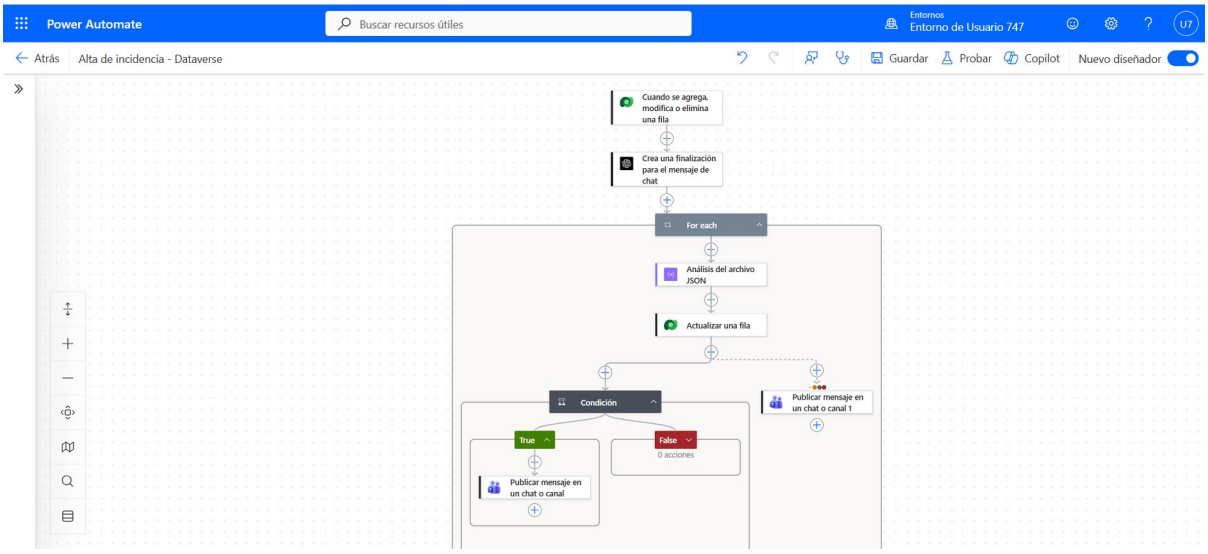



**Figura 8.** *Flujo 1: clasificación, condición de escalado y rama de error (línea discontinua).*

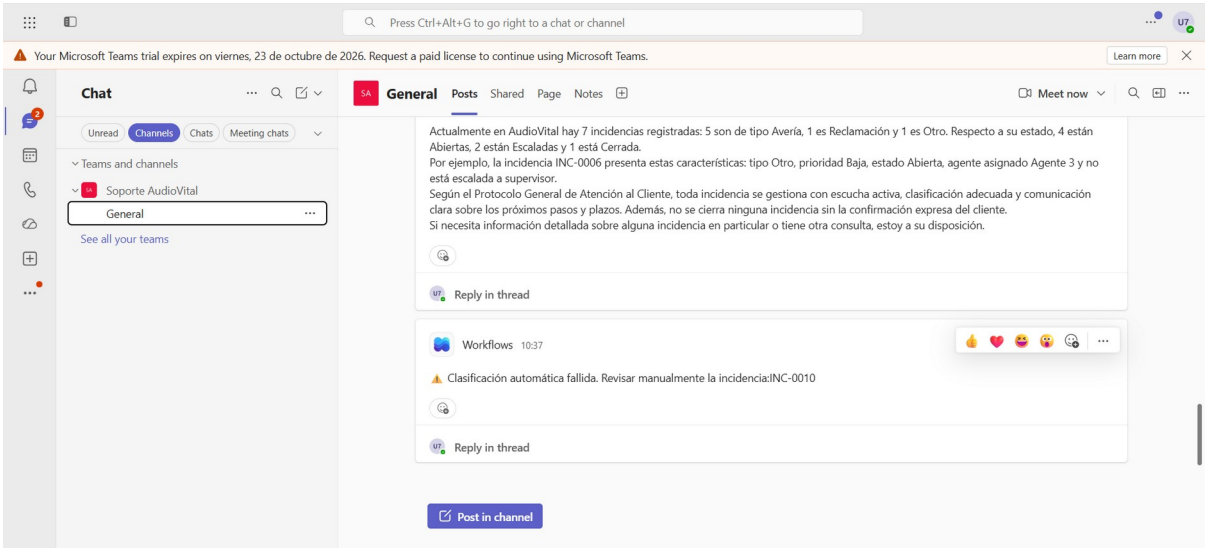



**Figura 9.** *Aviso de clasificación fallida durante la prueba de fallo forzado (INC-0010).*

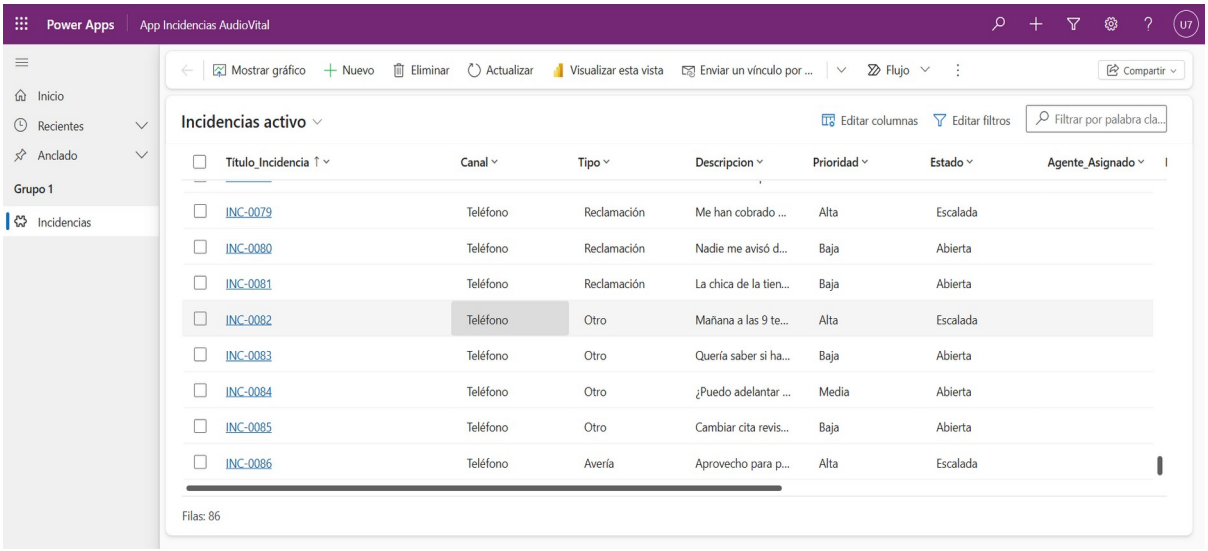



**Figura 10.** *Incidencias de prioridad Alta con Estado "Escalada".*

# 14. Continuación

La **Fase 3** abordaría la operación y el gobierno de la solución:

- Identidad de servicio en lugar de clave de API, con secretos en Azure Key Vault.
- Roles de seguridad de Dataverse (agente, supervisor, administrador) y RBAC por recurso con mínimo privilegio.
- Tabla de registro de ejecuciones en Dataverse (modelo, tokens, latencia, resultado, error) conectada al panel de Power BI.
- Diagnósticos del recurso de Azure OpenAI y alertas sobre fallos de los flujos, que habrían detectado la incidencia de la sección 10.

Más adelante, el mismo patrón se aplicaría a otros dos casos de uso ya identificados —solicitudes de compra con aprobación por importe y cualificación de clientes potenciales— para demostrar que la arquitectura es reutilizable.

> *Proyecto personal de portafolio desarrollado íntegramente por José Rafael Vázquez Silva. AudioVital es una marca ficticia; todos los datos son inventados.*

MEMORIA TÉCNICA

# Gestión de Incidencias con Power Platform e IA

Fase 3: operación, seguridad y gobierno — y hoja de ruta a producción

Azure Key Vault · Entra ID · RBAC · Log Analytics · Cost Management · Dataverse · Power Automate · Teams (tarjetas adaptables)

**José Rafael Vázquez Silva**
Proyecto personal de portafolio
Octubre de 2026

*AudioVital es una marca ficticia. Todos los clientes, incidencias y documentos son inventados para este proyecto y no corresponden a ninguna empresa real.*

# Índice

1. Resumen ejecutivo.....3
2. Punto de partida y objetivo de la Fase 3.....4
3. Modelo de uso: quién usa y quién gestiona la solución.....5
4. Arquitectura final (Fases 1 a 3).....6
5. Seguridad: Key Vault, entidad de servicio y RBAC.....7
6. Observabilidad y control de costes.....9
7. Incidente: bucle de autodisparo del Flujo 2.....11
8. Ciclo de vida de la incidencia en Dataverse.....14
9. Flujo 3 — Cierre con resolución obligatoria.....15
10. Flujo 4 — Asignación «Me encargo» con tarjeta adaptable.....17
11. Problemas resueltos en la Fase 3.....19
12. Lo que no se construyó.....20
13. Límites conocidos (consolidados).....21
14. Hoja de ruta para llevarlo a producción.....22
15. Evidencias.....25

# 1. Resumen ejecutivo

Este documento continúa la *Memoria técnica de las Fases 1 y 2* del proyecto **Gestión de Incidencias con Power Platform e IA** (marca ficticia **AudioVital**). Al cerrar la Fase 2, la solución clasificaba incidencias con Azure OpenAI y respondía en Teams fundamentándose en documentación corporativa (RAG con Azure AI Search). Funcionaba, pero tenía problemas de **operación y gobierno**: una clave de API pegada en un flujo, ninguna visibilidad sobre consumo, costes ni errores, y una incidencia que se creaba y clasificaba, pero no tenía dueño ni forma de cerrarse.

La **Fase 3** resuelve esos tres frentes:

| Frente | Antes (Fase 2) | Después (Fase 3) |
|---|---|---|
| Secretos | Clave de Azure AI Search escrita en el paso HTTP | Clave de **consulta** (solo lectura) guardada en **Azure Key Vault** y leída por una **entidad de servicio** con RBAC de mínimo privilegio |
| Observabilidad | Ninguna: la cuota semántica agotada se descubrió por casualidad | **Métricas** de tokens, **presupuesto con alertas** y **diagnósticos** de Azure OpenAI en **Log Analytics**, consultables con KQL |
| Coste y fiabilidad del asistente | Respuestas de hasta 5 min, duplicadas, errores 429 | Bucle de autodisparo cortado y carga del prompt reducida un **80 %**: una respuesta en 8–15 s |
| Ciclo de vida | Alta → clasificación → aviso. Sin dueño ni cierre | Autor y propietario reales, **asignación desde Teams** («Me encargo»), **cierre con resolución obligatoria** y aviso al autor |

Se construyeron **dos flujos nuevos** (cierre y asignación) y se modificaron los dos existentes. Todo está probado en el entorno real con evidencias (capturas 17 a 61). La memoria termina con lo que **no** se llegó a construir y con una **hoja de ruta concreta para llevar la solución a producción** en una organización real.

> *Proyecto personal de portafolio. Todos los datos, clientes, documentos y la marca AudioVital son ficticios. El entorno es real: un inquilino de formación con crédito de Azure temporal y una prueba de Microsoft Teams.*

# 2. Punto de partida y objetivo de la Fase 3

La pregunta de la Fase 3 fue: **¿qué le falta a esta solución para que un equipo de IT la pudiera operar con tranquilidad?** Del análisis salieron dos bloques, ordenados por la fecha de caducidad del crédito de Azure (~21/10/2026):

## Bloque A — Azure (seguridad, observabilidad y coste)

- Sacar la clave del flujo a **Azure Key Vault** y leerla con una identidad propia, no con la cuenta del autor.
- **RBAC por recurso** con el principio de mínimo privilegio.
- **Diagnósticos** de Azure OpenAI hacia Log Analytics, **métricas de tokens** y **presupuesto con alertas**.

## Bloque B — Power Platform (ciclo de vida de la incidencia)

- Sustituir el «agente asignado» escrito a mano por la **identidad real** de quien registra y de quien gestiona.
- Nuevos estados (*En curso*, *Pendiente de revisión*) y columna **Resolución**.
- **Cierre controlado**: no se puede cerrar sin resolución y el autor recibe el aviso.
- **Asignación** de las escaladas desde Teams con una tarjeta adaptable.

Durante el Bloque A apareció un incidente real (el bucle de autodisparo del Flujo 2, sección 7), que se trató como prioridad antes de seguir.

# 3. Modelo de uso: quién usa y quién gestiona la solución

La solución es una **herramienta interna**. El cliente final no entra en Power Apps ni en Teams: habla con la tienda o con atención telefónica. Así lo exige además el protocolo de atención de la base documental, que prohíbe exponer campos internos como la prioridad o el escalado.

| Rol | Qué hace en el sistema | Herramienta |
|---|---|---|
| Agente (tienda o teléfono) | Registra la incidencia con sus palabras; consulta al asistente dudas sobre garantías, citas o reclamaciones; cierra la incidencia con su resolución | Power Apps · Teams |
| Supervisor | Recibe las escaladas (prioridad Alta) y se las asigna con «Me encargo»; revisa clasificaciones fallidas | Teams (canal) · Power Apps |
| Responsable de zona / dirección | Sigue volumen, tipos, prioridades y estados | Power BI |

## Tres responsabilidades separadas

- **Atención al cliente es dueña del contenido:** mantiene los PDFs de políticas y los criterios de prioridad. Cambiar una política no exige tocar ningún flujo, solo volver a indexar.
- **IT / administración de Power Platform es dueña de la plataforma:** licencias, entornos, permisos, Key Vault, presupuesto y vigilancia de los flujos.
- **Calidad:** repite la evaluación de la clasificación con incidencias nuevas cada vez que cambia el prompt o el modelo.

## Principio de diseño: el aviso avisa, la lista de trabajo vive en Dataverse

Un mensaje de Teams se pierde entre otros. Por eso Teams solo **notifica** y permite **actuar** (botón «Me encargo»), pero el estado real de cada incidencia, su autor, su propietario y su resolución viven siempre en **Dataverse**, que es la fuente única de verdad.

# 4. Arquitectura final (Fases 1 a 3)

```
Power Apps ──► Dataverse (fuente única de verdad)
                  │
                  ├─► Flujo 1  Alta ──► Azure OpenAI (clasificación)
                  │       ├─► escribe Tipo, Prioridad, Estado, Fecha_Alta
                  │       └─► Teams: aviso de escalado / de fallo
                  ├─► Flujo 3  Cerrada ──► exige Resolución, Fecha_Cierre,
                  │                        aviso al autor por Teams
                  ├─► Flujo 4  Escalada ──► tarjeta «Me encargo»
                  │                        ──► Propietario + En curso
                  └─► Power BI

Teams ──► Flujo 2  Asistente
            ├─► Dataverse (50 últimas incidencias)
            ├─► Key Vault (entidad de servicio) ──► clave de consulta
            ├─► Azure AI Search (híbrida) ◄── Blob Storage + embeddings
            └─► Azure OpenAI ──► respuesta citando la fuente

Azure OpenAI ──► Diagnósticos ──► Log Analytics (KQL)
Cost Management ──► presupuesto y alertas
```

| Flujo | Desencadenador | Qué hace | Fase |
|---|---|---|---|
| 1 · Alta de incidencia | Fila agregada en Incidencias | Clasifica con IA (temperatura 0, JSON), escribe Tipo, Prioridad, Escalada, Estado y Fecha_Alta; avisa por Teams si es Alta o si falla | 1 · 2 · 3 |
| 2 · Consulta (asistente) | Palabra clave en el canal de Teams | Corta su propio eco, lee 50 incidencias, obtiene la clave de Key Vault, busca en AI Search y responde con Azure OpenAI citando la fuente | 1 · 2 · 3 |
| 3 · Cierre con aviso | Estado cambia a Cerrada | Si falta la resolución, devuelve a En curso; si no, fecha el cierre y avisa al autor por Teams | 3 |
| 4 · Escalado «Me encargo» | Estado cambia a Escalada | Publica una tarjeta con botón en el canal; quien pulsa pasa a ser propietario y la incidencia queda En curso | 3 |

## Recursos de Azure (grupo rg-incidencias-audiovital, East US)

| Recurso | Nombre | Función |
|---|---|---|
| Azure OpenAI | `openai-incidencias-rafael` | `gpt-4.1-mini` (100k TPM, 100 RPM) y `text-embedding-3-small` |
| Azure AI Search (Gratis) | `audiovital-search` | Índice `audiovital-rag` (11 fragmentos de 6 PDFs) |
| Storage Account (LRS) | `audiovitalstorage` | Contenedor privado `base-documental` |
| Key Vault (Estándar, RBAC) | `kv-audiovital-rvs` | Secreto `search-query-key` |
| Log Analytics | `log-audiovital` | Destino del diagnóstico `diag-openai` |
| Entra ID · registro de aplicación | `app-audiovital-flujos` | Entidad de servicio con la que el Flujo 2 lee Key Vault |
| Cost Management | `presupuesto-audiovital` | 25 €/mes con alertas |

# 5. Seguridad: Key Vault, entidad de servicio y RBAC

## 5.1 Problema

En la Fase 2 el paso HTTP del Flujo 2 llevaba la clave de Azure AI Search escrita en la cabecera `api-key`. Cualquiera con permiso de edición sobre el flujo, o acceso a su historial, podía leerla. Rotarla obligaba a editar el flujo.

## 5.2 Qué se construyó

1. **Registro de aplicación en Entra ID** (`app-audiovital-flujos`) con un secreto de cliente (`kv-lectura-flujos`, caduca el 13/1/2027). Es la identidad con la que el flujo lee el almacén: no depende de la cuenta personal del autor.
2. **Key Vault** `kv-audiovital-rvs`: plan Estándar, **modelo de permisos RBAC** (no directivas de acceso), retención de borrado temporal de 7 días.
3. **Asignaciones de rol con ámbito «este recurso»:** la cuenta del autor tiene *Agente de secretos de Key Vault* (crea y gestiona secretos); la aplicación tiene **solo** *Usuario de secretos de Key Vault* (lee secretos, nada más).
4. **Secreto `search-query-key`** con la **clave de consulta** de Azure AI Search, no la de administración: aunque se filtrara, solo permite buscar, no modificar ni borrar el índice.
5. **Flujo 2:** nueva acción **Obtener secreto** (conector Azure Key Vault, conexión `kv-audiovital-entidad-servicio` con autenticación de entidad de servicio) antes del paso HTTP. La cabecera `api-key` pasa a ser `body/value` del secreto.
6. **Entradas y salidas seguras** activadas en *Obtener secreto* y en *HTTP*: el historial de ejecuciones muestra «Contenido no mostrado debido a la configuración de seguridad».

## 5.3 Resultado

Prueba de punta a punta en Teams: *Obtener secreto* 0,7–0,8 s, *HTTP* 0,4–0,7 s y respuesta correcta del asistente. La clave ya no aparece en el flujo, en su definición ni en su historial.

## 5.4 Decisiones y compromisos

- **Clave de consulta en lugar de la de administración:** mínimo privilegio también en el secreto, no solo en quién lo lee.
- **Las salidas seguras se propagan:** como el resultado del paso HTTP es seguro, las entradas del paso de Azure OpenAI también salen ocultas. Ganamos seguridad, pero perdemos la visibilidad del prompt al depurar. Es un compromiso consciente.
- **No es una identidad administrada pura.** Los flujos de nube de Power Automate no tienen identidad administrada para este conector; la entidad de servicio usa un secreto de cliente, guardado cifrado en la conexión. Rotarlo es una tarea pendiente de automatizar.
- **Acceso público al Key Vault:** Power Automate no sale desde una IP fija, así que el almacén admite conexiones desde todas las redes, protegido por Entra ID y RBAC. En producción se cerraría (sección 14).

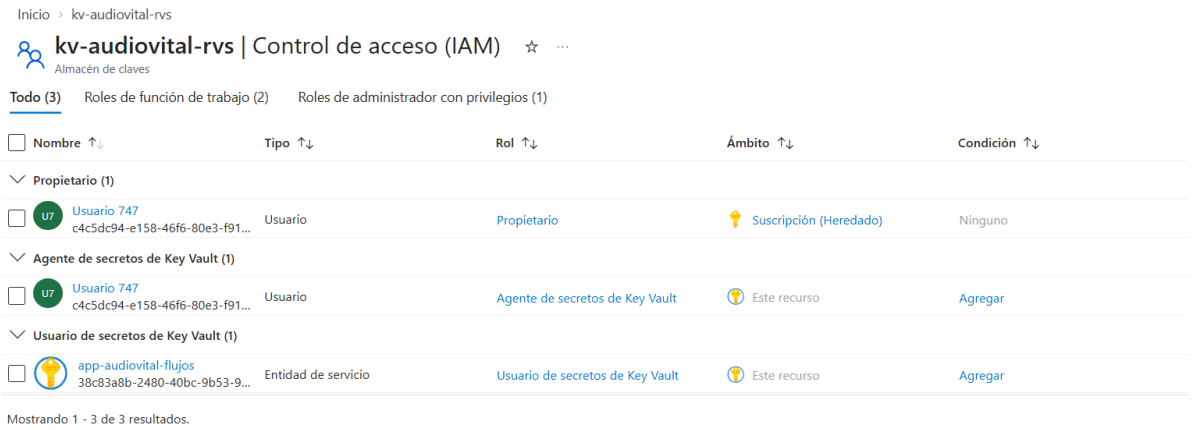



**Figura 1.** *Asignaciones de rol en el Key Vault: la aplicación solo puede leer secretos.*

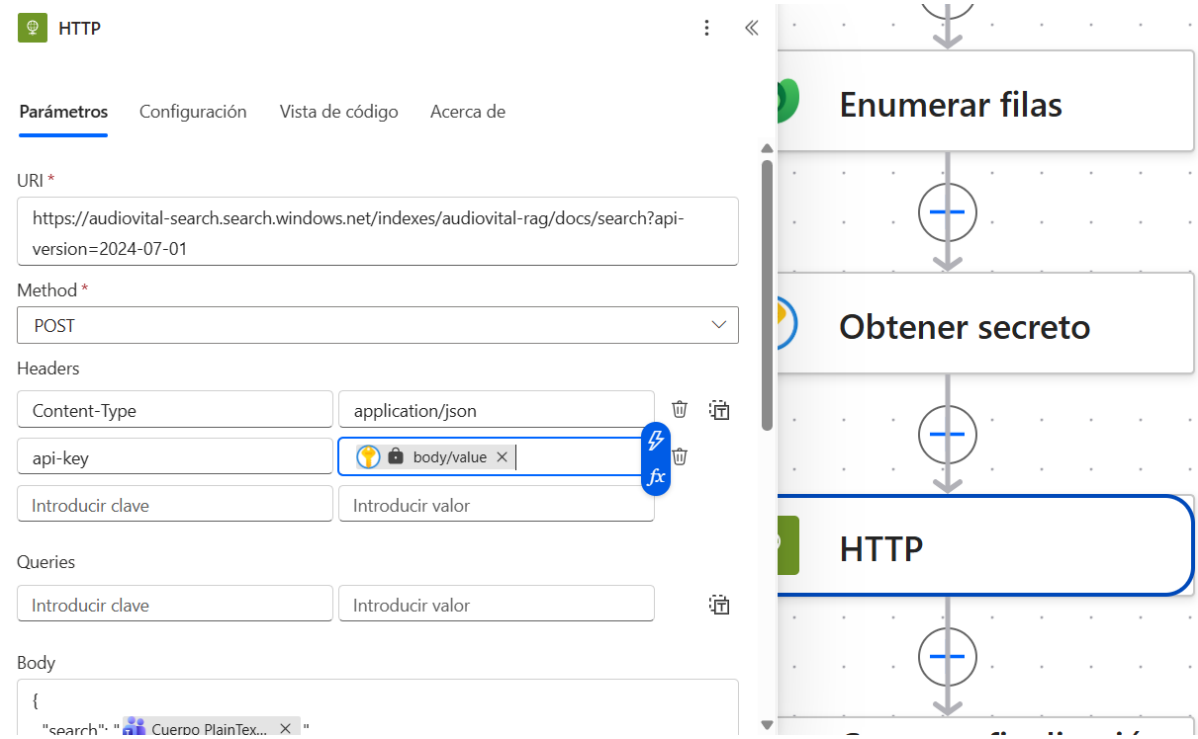



**Figura 2.** *Flujo 2: la cabecera api-key toma el valor del secreto leído de Key Vault.*

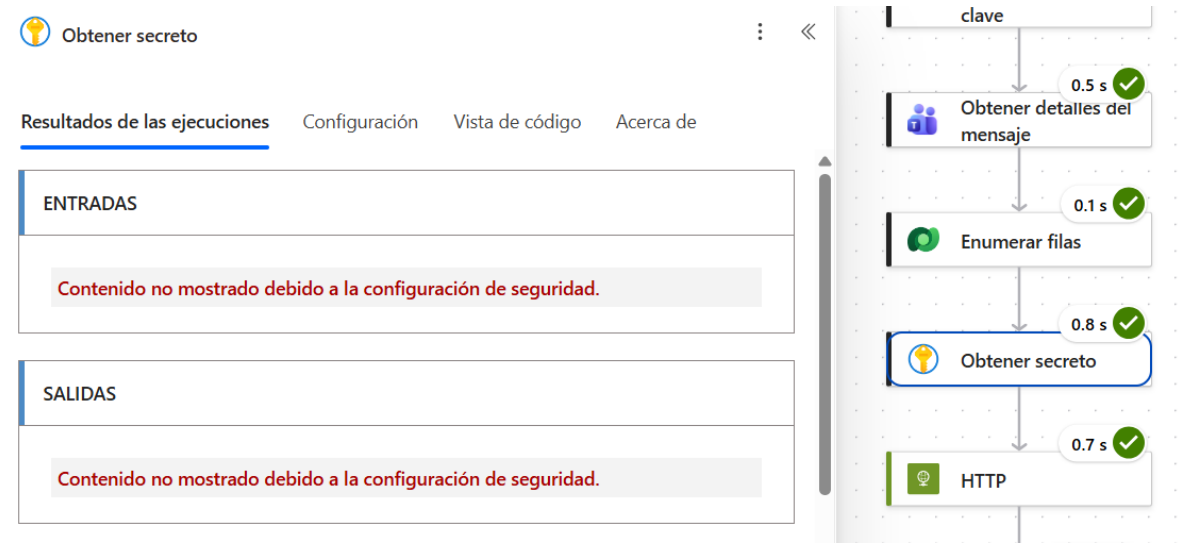



**Figura 3.** *Entradas y salidas seguras: el secreto no aparece en el historial.*

# 6. Observabilidad y control de costes

La Fase 2 dejó una lección: la cuota semántica gratuita se agotó y el asistente dejó de funcionar **sin que nadie lo supiera**. La Fase 3 añade tres capas para ver qué pasa y cuánto cuesta.

## 6.1 Métricas del recurso de Azure OpenAI

La métrica *Processed Inference Tokens* mostró el consumo real. Fue la que reveló el incidente de la sección 7: una meseta de ~150.000 tokens cada 5 minutos, **36,53 millones de tokens en 24 horas**, y la caída al corregirlo.

## 6.2 Coste por recurso y presupuesto

- **Análisis de costes** por recurso: octubre pasó de 9,62 € a 19,55 €, casi todo en Azure OpenAI. Key Vault, Storage y AI Search: céntimos o cero.
- **Presupuesto `presupuesto-audiovital`**: 25 €/mes, alerta por correo al **80 % del gasto real** y al **100 % del previsto**. Un bucle como el de la sección 7 habría disparado la alerta en lugar de descubrirse revisando métricas.

## 6.3 Diagnósticos en Log Analytics

Configuración de diagnóstico `diag-openai` en el recurso de Azure OpenAI, enviando las categorías **Audit**, **Request and Response** y **Azure OpenAI Request Usage**, además de **AllMetrics**, al área de trabajo `log-audiovital`. Consulta de verificación:

```
AzureDiagnostics
| where TimeGenerated > ago(1h)
| project TimeGenerated, OperationName, ResultSignature, DurationMs
| order by TimeGenerated desc
```

Resultado: una pregunta en Teams aparece como `post` (200) → `Embeddings_Create` (200, 175 ms) → `chatcompletions_create` (200, 1.079 ms). Es decir, se ve **cada llamada del RAG**: la vectorización de la consulta y la generación de la respuesta, con su código y su latencia. Los 429 de la sección 7 habrían aparecido aquí con `ResultSignature = 429`.




**Figura 4.** *Log Analytics: cada pregunta genera una llamada de embeddings y otra de chat, ambas con código 200.*

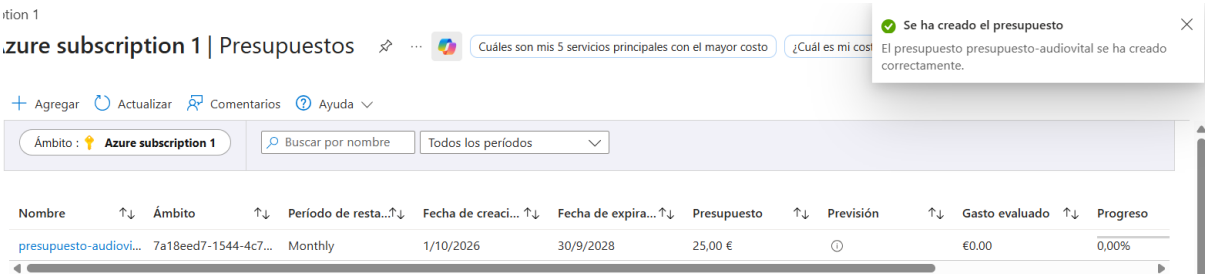



**Figura 5.** *Presupuesto mensual de 25 € con alertas por correo.*

# 7. Incidente: bucle de autodisparo del Flujo 2

## 7.1 Síntoma

Desde la tarde del 15/10 el asistente tardaba hasta **5 minutos** en contestar y a veces respondía **dos o tres veces**. El historial mostraba ejecuciones cada ~5 minutos, también de noche, sin que nadie escribiera en el canal.

## 7.2 Diagnóstico, paso a paso

1. El **historial de reintentos** del paso de Azure OpenAI mostraba un primer intento con **429** (demasiadas peticiones) y un segundo correcto tras la espera automática de Power Automate: de ahí los 5 minutos.
2. Se comprobó el **límite del despliegue**: 100.000 tokens y 100 peticiones por minuto. No era bajo, así que el límite no era la causa sino el síntoma.
3. El desencadenador escucha la palabra clave **«Incidencias»**… y las respuestas del asistente también la contienen. **El asistente se disparaba a sí mismo.**
4. Confirmado en la salida de *Obtener detalles del mensaje*: `from.user = null` y `from.application.displayName = "Workflows"`, con `applicationIdentityType = "bot"`.
5. Agravante: *Enumerar filas* sin filtro enviaba **247.625 bytes** (~60–70k tokens) en cada llamada. Dos llamadas en el mismo minuto superaban los 100k TPM.

## 7.3 Corrección

1. Flujo **desactivado** de inmediato para cortar el gasto.
2. **Condición antibucle** justo después de *Obtener detalles del mensaje*: `empty(body('Obtener_detalles_del_mensaje')?['from']?['user'])` es igual a `true` → **Finalizar (correcto)**. Si el mensaje no lo escribe una persona, el flujo termina sin llamar a la IA.
3. **Enumerar filas filtrado:** solo 12 columnas necesarias, orden `createdon desc` y **50 filas** como máximo.

## 7.4 Resultado

| Medida | Antes | Después |
|---|---|---|
| Tiempo de respuesta | ~5 min (con 429) | **8–15 s** |
| Respuestas por pregunta | 2–3 | **1** |
| Autodisparo | Llamaba a la IA cada vez | Termina en < 1 s sin llamar a la IA |
| Carga de Enumerar filas | 247.625 bytes | **48.387 bytes (−80 %)** |
| Tokens | 36,53 M en 24 h | Caída inmediata a niveles normales |

## 7.5 Lección

Un 429 no siempre significa «sube la cuota». Aquí la causa era de **diseño** (un desencadenador que oye sus propias respuestas) y de **tamaño del prompt** (enviar la tabla entera). Corregir la causa fue gratis; subir la cuota habría ocultado el problema y multiplicado el coste. Además, sin métricas ni presupuesto el bucle habría seguido consumiendo crédito en silencio: por eso las piezas de la sección 6 no son decorativas.




**Figura 6.** *El mensaje que dispara el flujo lo envió el bot Workflows: from.user es null.*




**Figura 7.** *Tokens procesados: meseta cada 5 minutos durante el bucle y caída tras la corrección.*

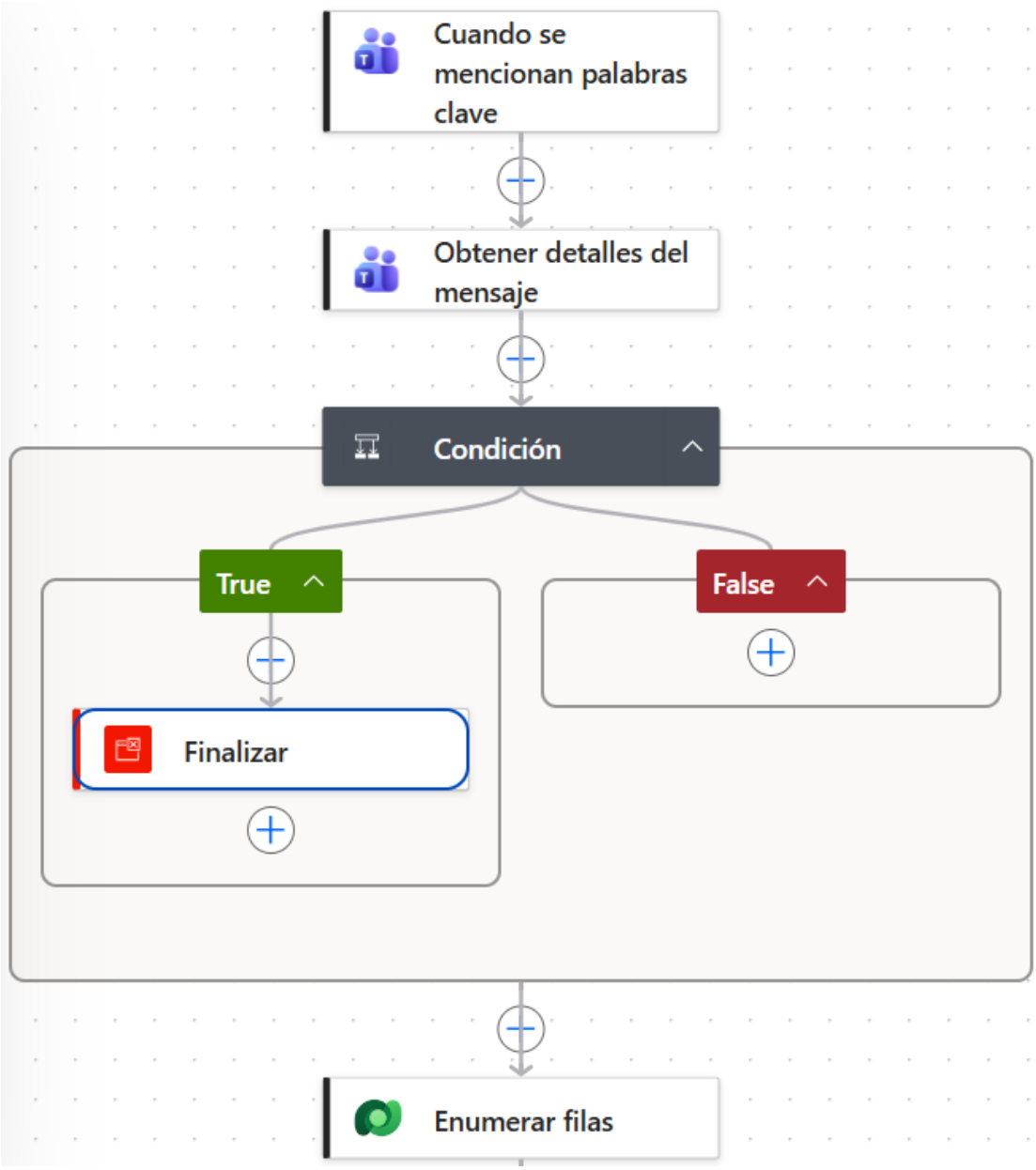



**Figura 8.** *Condición antibucle: si no escribe una persona, el flujo termina.*

# 8. Ciclo de vida de la incidencia en Dataverse

## 8.1 De «agente asignado» a identidad real

La Fase 1 usaba una columna de opción *Agente asignado* (Agente 1/2/3) que se rellenaba a mano: no identificaba a nadie y no servía para permisos. En la Fase 3 se sustituye por las columnas de sistema de Dataverse:

- **Creado por (Autor):** quien registra la incidencia. Automático e inmutable. Se muestra en el formulario como campo de solo lectura.
- **Propietario:** quien la gestiona. Es la columna que usan los roles de seguridad de Dataverse y la que cambia el botón «Me encargo».

La columna antigua se quitó del formulario, pero se mantiene en la tabla por las incidencias históricas.

## 8.2 Estados y columnas nuevas

| Estado | Valor | Quién lo escribe |
|---|---|---|
| Abierta | 610150000 | Valor por defecto de la columna; Flujo 1 si la prioridad no es Alta |
| En curso | 610150001 | Flujo 4 (al asignarse) o Flujo 3 (si se intenta cerrar sin resolución) |
| Escalada | 610150002 | Flujo 1 cuando la prioridad es Alta |
| Cerrada | 610150003 | El agente, desde la app (solo se mantiene si hay resolución) |
| Pendiente de revisión | 610150005 | Creado para la rama de error del Flujo 1 (conexión pendiente, sección 12) |

- **Resolución:** texto de varias líneas, en el formulario junto a Fecha_Cierre.
- **Fecha_Alta:** ahora la escribe el Flujo 1 en la misma actualización que la clasificación, con `formatDateTime(triggerOutputs()?['body/createdon'], 'yyyy-MM-dd')`. Antes quedaba vacía. Verificado con INC-0087.

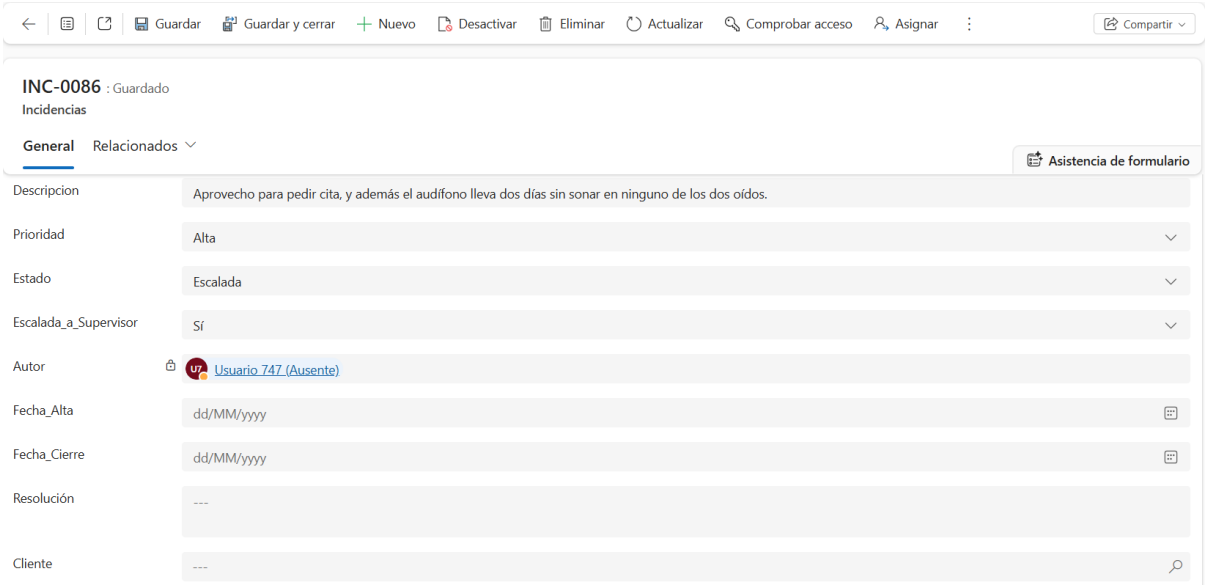



**Figura 9.** *Ficha de la incidencia con Autor (solo lectura), Fecha_Cierre y Resolución.*

# 9. Flujo 3 — Cierre con resolución obligatoria

## 9.1 Objetivo funcional

Que ninguna incidencia se cierre sin dejar escrito **cómo se resolvió**, que la fecha de cierre sea fiable y que quien la registró **se entere**, para poder informar al cliente. El aviso directo al cliente queda fuera a propósito: requiere otro canal y otras garantías de protección de datos.

## 9.2 Diseño

- **Desencadenador:** Dataverse, tipo de cambio *Actualizar*, tabla Incidencias, ámbito Organización, **Seleccionar columnas** `cr3f3_estado` y **Filtrar filas** `cr3f3_estado eq 610150003`. Solo arranca cuando el *Estado* cambia a Cerrada, no con cualquier edición.
- **Condición:** `empty(triggerOutputs()?['body/cr3f3_resolucion'])` es igual a `true`.
- **Rama sin resolución:** *Actualizar una fila* → Estado = **En curso**. La incidencia «rebota».
- **Rama con resolución:** *Actualizar una fila* → `Fecha_Cierre = utcNow()`; *Obtener una fila por id* en la tabla Usuarios con el `_createdby_value` del desencadenador (columnas `internalemailaddress,fullname`); Teams *Publicar mensaje* (Bot de Flow → chat con el autor): «✅ Incidencia «INC-xxxx» cerrada. Resolución: … Informa al cliente del resultado».

## 9.3 Por qué no se crea un bucle

El flujo modifica la misma fila que lo dispara, pero el desencadenador solo vigila `cr3f3_estado` y solo cuando vale *Cerrada*. Escribir *En curso* o *Fecha_Cierre* no cumple el filtro. **Filtrar en el desencadenador** (y no con una condición dentro del flujo) evita además ejecuciones inútiles.

## 9.4 Pruebas con INC-0086

| Prueba | Resultado |
|---|---|
| Cerrada **sin** resolución (11:21) | Condición `true` → vuelve a *En curso*. Correcto en 601 ms |
| Cerrada **con** resolución (13:04) | Condición `false` → Fecha_Cierre 19/10/2026, autor localizado y aviso en Teams. Correcto en 3 s |

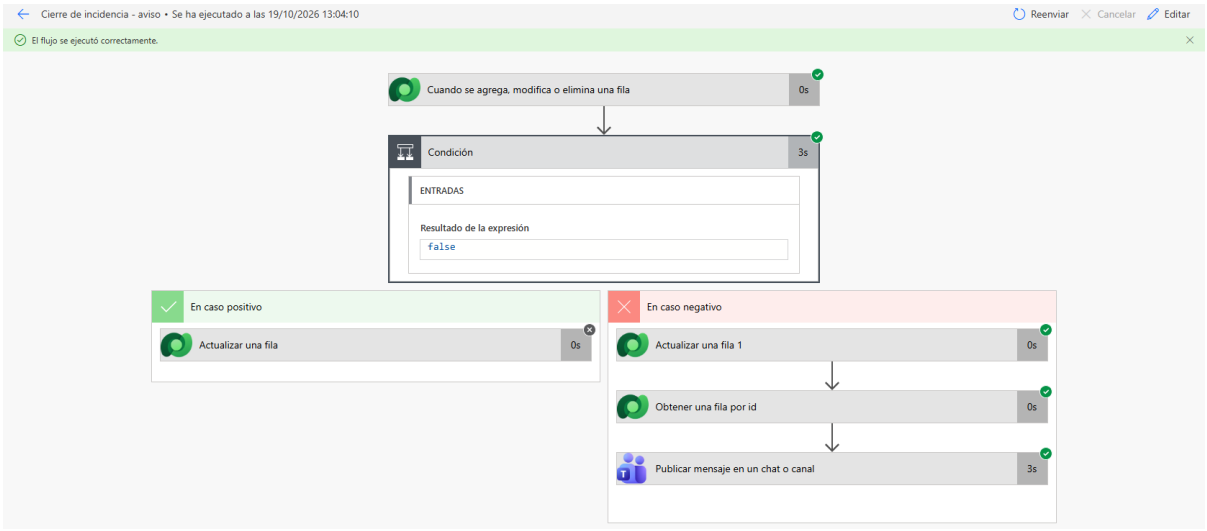



**Figura 10.** *Ejecución con resolución: fecha de cierre, lectura del autor y aviso por Teams.*

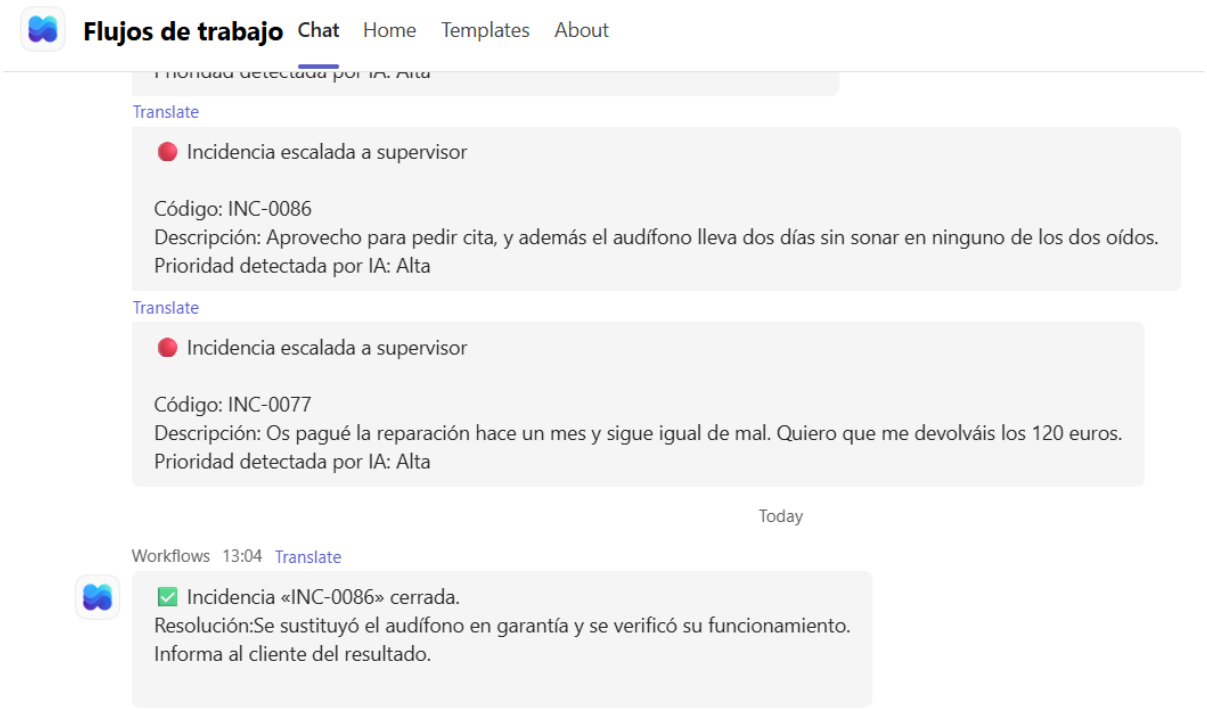



**Figura 11.** *Ciclo completo en Teams: aviso de escalado (Flujo 1) y aviso de cierre (Flujo 3).*

# 10. Flujo 4 — Asignación «Me encargo» con tarjeta adaptable

## 10.1 Objetivo funcional

Que una escalada no dependa de que el supervisor lea un mensaje y vaya a la aplicación. La tarjeta permite **hacerse cargo con un clic**, y el sistema registra **quién** se encargó y **cuándo** pasó a estar en curso.

## 10.2 Por qué un flujo separado

*Publicar tarjeta adaptable y esperar una respuesta* deja la ejecución **esperando** hasta que alguien pulsa, lo que pueden ser horas. Meterlo en el Flujo 1 bloquearía la clasificación. Separarlo además desacopla: el Flujo 4 reacciona a *Escalada* venga de donde venga (IA o cambio manual).

## 10.3 Diseño

1. **Desencadenador:** Dataverse, *Actualizar*, Seleccionar columnas `cr3f3_estado`, Filtrar filas `cr3f3_estado eq 610150002`.
2. **Teams — Publicar tarjeta adaptable y esperar una respuesta** en el canal *Soporte AudioVital / General*. Tarjeta Adaptive Card 1.4 con título, código, prioridad, descripción y un `Action.Submit` «🙋 Me encargo». Al responder, la tarjeta se sustituye por «✅ Asignada. Gestión en curso».
3. **Enumerar filas** en Usuarios con `domainname eq '<responder/userPrincipalName>'` y `1` fila: busca en Dataverse a quien pulsó.
4. **Actualizar una fila:** Estado `=` En curso y Propietario `=` `concat('systemusers(', first(outputs('Enumerar_filas')?['body/value'])?['systemuserid'], ')')`.

## 10.4 Error encontrado y corregido

La primera ejecución falló en *Actualizar una fila* con **BadRequest**: `The supplied reference link -- systemusers() -- is invalid`. *Enumerar filas* había terminado «correctamente»… pero **sin resultados**. El filtro buscaba por `internalemailaddress` con el correo de la respuesta, y Teams identifica a quien responde por su **UPN** (nombre de inicio de sesión). Se cambió a `domainname` (el campo que guarda el UPN en Dataverse) y `responder/userPrincipalName`. Tras **Reenviar**: todo en verde (tarjeta 12 s, búsqueda 0,1 s, actualización 0,5 s) e INC-0087 pasó de *Escalada* a *En curso* con su propietario.

> *Lección: en automatización, un paso «correcto» puede devolver cero filas. Antes de usar first() sobre una consulta conviene comprobar que no viene vacía y tener una ruta alternativa.*

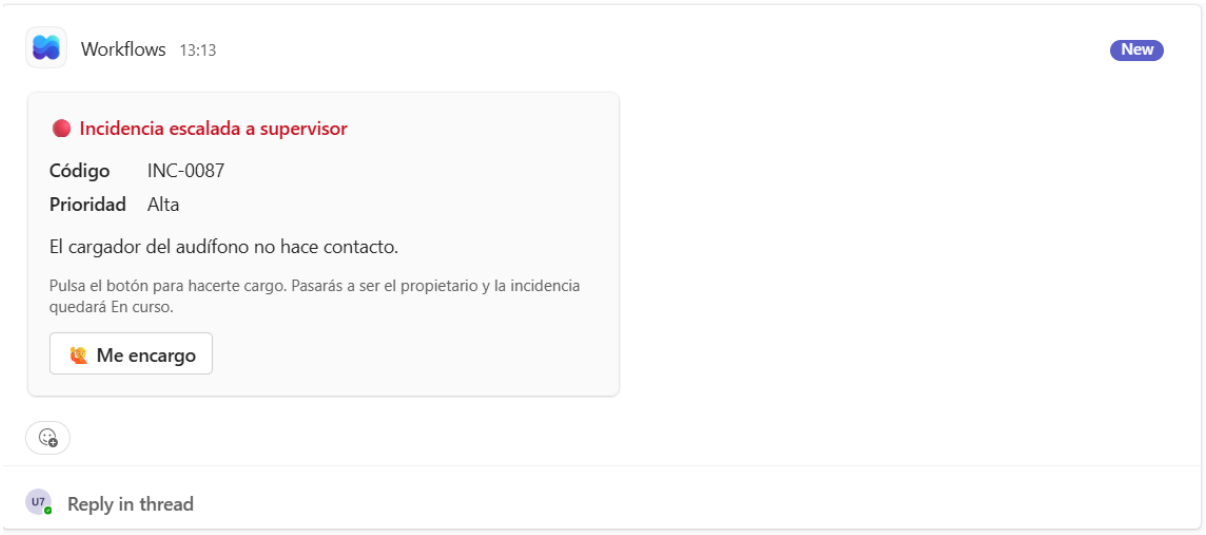



**Figura 12.** *Tarjeta adaptable publicada en el canal al escalar INC-0087.*

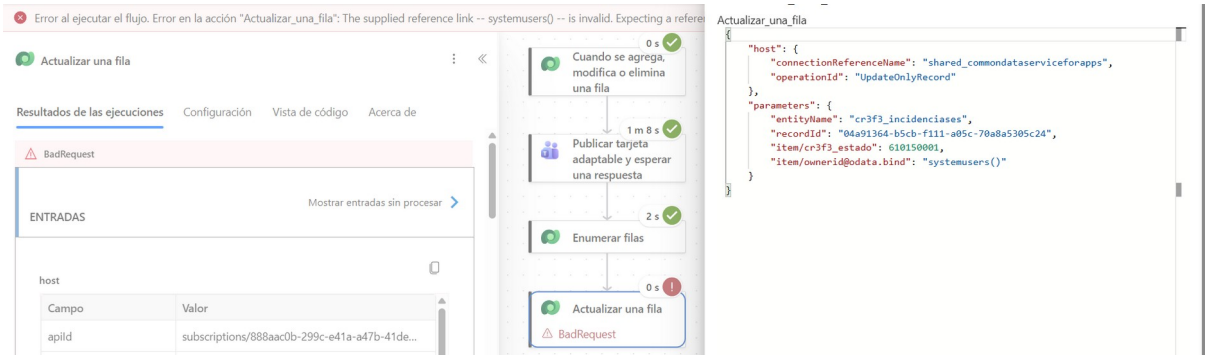



**Figura 13.** *Diagnóstico del BadRequest: el propietario llegó vacío porque la búsqueda no encontró al usuario.*




**Figura 14.** *INC-0087 en curso tras pulsar «Me encargo».*

# 11. Problemas resueltos en la Fase 3

| Problema | Causa | Solución |
|---|---|---|
| Conexión a Key Vault rota tras crearla | Se pegó el *Id.* del secreto de cliente en lugar de su *Valor* | Reparar la conexión con el Valor (solo se muestra al crearlo) |
| El flujo parecía no haberse guardado | Guardado interrumpido | «Recuperar flujo» y volver a guardar |
| Respuestas de 5 min y 429 | Bucle de autodisparo + prompt de 247 KB | Condición antibucle y Enumerar filas filtrado (sección 7) |
| Respuestas duplicadas o triplicadas | El mismo bucle | Ídem |
| El asistente atribuía incidencias a la clienta como si las hubiera «registrado» | El prompt no explica la diferencia Cliente / Creado por / Propietario | Detectado; ajuste del prompt pendiente (sección 12) |
| Valor `true` de una condición tratado como texto | Escrito a mano en lugar de como expresión | Introducirlo con fx |
| «Recipient» de Teams no admitía una expresión | Campo en modo selector de personas | ⚙️ → «Usar contenido dinámico» |
| La primera ejecución del Flujo 3 tardó en aparecer | Registro inicial del desencadenador de Dataverse | Esperar o desactivar/activar el flujo |
| Fecha_Alta vacía en todas las incidencias | Ningún flujo la escribía | Flujo 1: `formatDateTime(createdon)` en la misma actualización |
| Dos columnas llamadas «Estado» | Estado propio y *statecode* del sistema (Activo/Inactivo) | Identificar la correcta por sus opciones |
| BadRequest `systemusers()` | Búsqueda por correo; Teams devuelve el UPN | Buscar por `domainname` con `userPrincipalName` |
| Correo personal visible en capturas | Cabeceras y conexiones del portal | Revisión con OCR y tachado antes de publicar |

# 12. Lo que no se construyó

Parte del alcance previsto para la Fase 3 no se ejecutó antes de cerrar el proyecto (caducidad del crédito de Azure el ~21/10 y de la prueba de Teams el 23/10). Se declara para que no se dé por hecho:

| Pieza prevista | Estado | Cómo se haría |
|---|---|---|
| Rama de error del Flujo 1 → *Pendiente de revisión* | Estado creado; la rama sigue solo avisando por Teams | Añadir *Actualizar una fila* (Estado 610150005) en la rama «Ejecutar después de: ha fallado» |
| Recordatorio de escaladas sin asignar fuera de plazo | No construido | Flujo programado cada hora: Escaladas con `modifiedon` > plazo del protocolo → aviso al responsable superior |
| Evitar el doble aviso de escalado | Flujo 1 (texto) y Flujo 4 (tarjeta) avisan los dos | Quitar el mensaje de texto de la rama de escalado del Flujo 1 |
| Comprobar si la búsqueda del Flujo 4 viene vacía | No construido | Condición `empty()` antes de actualizar y aviso al canal |
| Roles de seguridad de Dataverse | No construido | Agente (crear y ver lo suyo), Supervisor (ver todo y asignar), Administrador |
| Tabla `Registro_Ejecuciones` | No construido; sustituido en parte por Log Analytics | Una fila por ejecución: incidencia, modelo, tokens, latencia, resultado, error |
| Auditoría de Dataverse | Desactivada | Activar en el entorno y en la tabla Incidencias |
| Bandeja del supervisor y Power BI ampliado | No construido | Vista «Pendientes» y página de tiempos de asignación y resolución |
| Semántica de columnas en el prompt del Flujo 2 | Detectado, no corregido | Explicar Cliente / Creado por / Propietario en el mensaje de sistema |

# 13. Límites conocidos (consolidados)

## Del entorno

- Inquilino de formación con **un solo usuario**: la asignación y los avisos funcionan con identidades reales, pero no se ha podido demostrar con varios agentes y supervisores.
- **Infraestructura temporal:** crédito de Azure hasta ~21/10/2026 y prueba de Teams hasta 23/10/2026. Después, la solución queda documentada y demostrable con evidencias, pero no ejecutable. El entorno de Power Platform (tipo Desarrollador) no caduca.
- **Azure AI Search en plan Gratis:** sin reordenación semántica (cuota agotada en la Fase 2), sin réplicas ni SLA.
- El modelo `gpt-4.1-mini` está en estado **Legacy** y se retira el 14/4/2027.

## De seguridad

- Entidad de servicio con **secreto de cliente**, no identidad administrada; rotación manual.
- Key Vault con **acceso público** (protegido por Entra ID y RBAC, sin punto de conexión privado).
- Las **salidas seguras** ocultan el prompt al depurar.
- Los flujos se ejecutan con conexiones de la cuenta del autor, no con una cuenta de servicio.

## Funcionales

- El asistente solo ve las **50 incidencias más recientes**: los recuentos globales («¿cuántas…?») pueden ser incompletos.
- Doble aviso de escalado (texto y tarjeta) hasta retirar el del Flujo 1.
- Evaluación de la clasificación con 45 casos de un solo autor (ver memoria de las Fases 1 y 2).

# 14. Hoja de ruta para llevarlo a producción

Lo construido demuestra el patrón completo, pero un despliegue real en una organización exigiría lo siguiente. Se ordena por áreas y, al final, en fases.

## 14.1 Gestión del ciclo de vida (ALM)

- **Tres entornos** de Power Platform —Desarrollo, Pruebas y Producción— como **entornos administrados**, con la aplicación, las tablas y los cuatro flujos dentro de una **solución**.
- **Solución no administrada en Desarrollo, administrada en Pruebas y Producción**, desplegada con **Power Platform Pipelines** o con GitHub Actions / Azure DevOps (Power Platform Build Tools). La solución se exporta y se versiona en el repositorio.
- **Referencias de conexión y variables de entorno** para todo lo que cambia entre entornos: URL e índice de AI Search, nombre del Key Vault, despliegue de OpenAI, equipo y canal de Teams. Nada escrito a mano en los flujos.
- **Variables de entorno de tipo secreto vinculadas a Azure Key Vault**, en lugar de la acción *Obtener secreto* con una entidad de servicio propia.

## 14.2 Identidad, acceso y seguridad

- Flujos propiedad de una **cuenta de servicio** (o usuario de aplicación de Dataverse), con copropietarios del equipo de IT. Ninguna conexión a nombre de una persona.
- **Roles de seguridad de Dataverse** (Agente, Supervisor, Administrador) y **unidades de negocio** por tienda o zona, para que cada agente vea solo lo suyo y el supervisor lo de su zona.
- **Grupos de Entra ID** para asignar roles, MFA y acceso condicional.
- **Azure:** Key Vault y AI Search con **punto de conexión privado** y red virtual (compatibilidad de Power Platform con redes virtuales en entornos administrados); acceso a AI Search con **RBAC** (*Search Index Data Reader*) en lugar de claves; **identidades administradas** entre AI Search, Storage y Azure OpenAI para el indexador; rotación automatizada de secretos o uso de certificados.
- **Directivas de prevención de pérdida de datos (DLP)** que solo permitan los conectores aprobados (Dataverse, Teams, Azure OpenAI, Key Vault, HTTP restringido).

## 14.3 Datos personales y cumplimiento

- Las descripciones de incidencias pueden contener **datos de salud** (pérdida de audición): categoría especial del RGPD. Requiere **evaluación de impacto (EIPD)**, base legal, minimización y retención definida.
- Despliegues de Azure OpenAI en **zona de datos de la UE** y recursos en regiones europeas (hoy todo está en East US).
- **Minimizar lo que se envía al modelo:** no la tabla de incidencias, sino solo los campos imprescindibles; nunca datos de contacto del cliente.
- **Auditoría de Dataverse** activada y política de retención de los registros de diagnóstico.

## 14.4 IA generativa y RAG

- **Modelo vigente** (no Legacy) y despliegue con la cuota adecuada; valorar capacidad aprovisionada si el volumen lo justifica.

- **Azure AI Search en plan Básico o Estándar**, con **reordenación semántica** de pago y al menos **2 réplicas** (requisito del SLA de lectura; 3 para lectura y escritura).
- **Indexación programada** o por eventos cuando cambie un documento; estrategia de fragmentación con solapamiento; **metadatos** (versión, fecha de vigencia, departamento) y **filtros de seguridad** si hay documentos restringidos.
- **Citas con enlace** al documento y a su versión, para que el agente pueda comprobar la fuente.
- **Evaluación continua:** el conjunto de 45 casos como prueba de regresión antes de cambiar prompt o modelo, ampliado con incidencias reales anonimizadas y un segundo anotador. Para el RAG, medir la recuperación (si el documento correcto está entre los *k* primeros) y la fidelidad de la respuesta a la fuente.
- **Prompts versionados** en la solución, con pruebas antes de publicarlos.
- Sustituir el asistente por palabra clave por un **agente de Copilot Studio** publicado en Teams, con Dataverse y AI Search como herramientas: consultas filtradas en lugar de las 50 últimas filas.

## 14.5 Fiabilidad y operación

- Patrón **try/catch con ámbitos** en todos los flujos, rama de error que deja la incidencia en *Pendiente de revisión* y aviso al equipo de soporte.
- **Control de simultaneidad** y comprobaciones de vacío antes de cada `first()`; paginación y consultas filtradas en lugar de listas completas.
- **Alertas** sobre Log Analytics (errores 429, latencia, fallos de llamadas) y sobre ejecuciones fallidas de flujos; tabla `Registro_Ejecuciones` para métricas de negocio en Power BI.
- Revisar los **límites de peticiones de Power Platform** por licencia y el **coste** de Azure con presupuestos por entorno.

## 14.6 Licencias y coste

- Los conectores **HTTP, Azure OpenAI y Key Vault son premium**: Power Automate Premium por usuario, o licencia de proceso para los flujos de la cuenta de servicio; Power Apps por usuario o por aplicación.
- Teams para todos los agentes y supervisores; Power BI Pro (o capacidad) para compartir el panel.
- Azure: OpenAI por tokens (céntimos por incidencia con prompts acotados), AI Search de pago por réplica, Key Vault y Log Analytics de coste bajo.

## 14.7 Adopción

- **Piloto en 2–3 tiendas** durante unas semanas, comparando la prioridad del sistema con la del supervisor y midiendo el tiempo hasta asignación y hasta cierre.
- El supervisor puede **corregir** la clasificación; las correcciones alimentan la evaluación.
- Formación breve, guía de uso y canal para reportar errores; responsables claros (contenido, plataforma, calidad).

## 14.8 Orden propuesto

| Etapa | Contenido | Criterio para pasar a la siguiente |
|---|---|---|
| 1 · Bases | Entornos, solución, cuenta de servicio, variables de entorno, DLP, roles | Despliegue reproducible de Desarrollo a Pruebas sin tocar flujos a mano |
| 2 · Seguridad y datos | EIPD, región UE, red privada, RBAC en AI Search, auditoría | Visto bueno de seguridad y protección de datos |

| Etapa | Contenido | Criterio para pasar a la siguiente |
|---|---|---|
| 3 · Calidad | Modelo vigente, búsqueda semántica, evaluación ampliada, alertas, try/catch | Acierto y escalados perdidos dentro del umbral acordado |
| 4 · Piloto | 2–3 tiendas con supervisión humana de la clasificación | KPIs del piloto y satisfacción de agentes y supervisores |
| 5 · Despliegue | Resto de tiendas por oleadas, formación y soporte | Operación estable y revisiones periódicas de calidad |

# 15. Evidencias

Las 45 capturas de la Fase 3 están en `capturas/fase3/` del repositorio, con su índice. Todas se revisaron antes de publicar: ningún secreto visible y los correos de la cuenta tachados.

| Capturas | Tema |
|---|---|
| 17 – 26 | Entra ID, Key Vault, RBAC, claves de consulta, conexión con entidad de servicio y prueba en Teams |
| 27 – 29 | Respuesta de 5 minutos, salidas seguras e historial de reintentos con 429 |
| 30 – 38 | Límite del despliegue, bucle de autodisparo, condición antibucle, Enumerar filas filtrado y respuestas corregidas |
| 39 – 43 | Costes por recurso, métricas de tokens, presupuesto, diagnóstico y consulta KQL en Log Analytics |
| 44 – 45 | Formulario con Autor y Resolución |
| 46 – 53 | Flujo 3: configuración, pruebas sin y con resolución y avisos en Teams |
| 54 – 55 | Fecha_Alta automática en el Flujo 1 |
| 56 – 61 | Flujo 4: tarjeta «Me encargo», error de la búsqueda por correo, corrección y resultado |

> *Proyecto personal de portafolio desarrollado por José Rafael Vázquez Silva con asistencia de una herramienta de IA para el diseño guiado y la documentación. Todas las configuraciones se hicieron, probaron y verificaron en el entorno real.*